

分类号: _____

密级: _____

UDC: _____

编号: _____

工学硕士学位论文

面向深度学习的 GPU 访存优化研究

硕士研究生: 吴文文

指导教师: 张国印 教授

学科、专业: 计算机科学与技术

哈尔滨工程大学

2019 年 3 月

分类号：_____

密 级：_____

UDC：_____

编 号：_____

工学硕士学位论文

面向深度学习的 GPU 访存优化研究

硕 士 研 究 生：吴文文

指 导 教 师：张国印 教授

学 位 级 别：工学硕士

学 科 、 专 业：计算机科学与技术

所 在 单 位：计算机科学与技术学院

论 文 提 交 日 期：2019 年 1 月

论 文 答 辩 日 期：2019 年 3 月

学 位 授 予 单 位：哈尔滨工程大学

Classified Index:

U.D.C:

A Dissertation for the Degree of M. Eng

Research on GPU Memory Optimization for Deep Learning

Candidate: Wu Wenwen

Supervisor: Prof. Zhang Guoyin

Academic Degree Applied for: Master of Engineering

Specialty: Computer Science and Technology

Date of Submission: Jan. , 2019

Date of Oral Examination: Mar. , 2019

University: Harbin Engineering University

摘 要

在人工智能时代的大背景下，深度学习算法的应用越来越广，研究者对深度学习模型的精度要求和性能要求也越来越高，训练不断加深的神经网络和多特征的大数据，在提高精度的同时牺牲了算法的计算性能。虽然 GPU 高速并行加速设备和 CuDnn 等加速库分别从硬件和算法方面优化了深度学习性能，但是，在诸多优化方法的实现中，如何能够利用 GPU 自身有限的内存空间发挥其高效的计算能力仍然是研究领域中的一个很大的挑战。

本文在深入了解 GPU 内存架构的研究基础上，针对基于卷积的神经网络模型，提出了一种基于数据运算代价的数据流交换方法来优化 GPU 访存性能。本文的主要工作如下：

研究了新一代 GPU Pascal 架构的内存特征，得出优化程序访存的编程方法和 GPU 内存参数，主要包括缓存特征参数，缓存策略，全局内存吞吐量和各级内存访问时延。本文将细粒度方法和嵌入汇编语法的基准测试方法相结合，能够更加全面的发现了各级内存的使用限制条件，同时为下一步数据运算模型的建模提供了重要的特征参数。

提出了面向深度学习的 GPU 微架构访存性能评估方法。在内存特征参数的基础上，研究了 GPU 上数据传输和运算代价的建模问题，包括基于 Roofline 的运算峰值评估模型、基于 LogGp 的 GPU-CPU 之间数据传输模型和预运行分析方法，为下一步应用于 TensorFlow 框架的内存优化提供了依据。

提出了基于数据运算代价的数据流交换方法。结合之前的运算代价模型研究，在理论模型和实际运行性能的支持下，在现有的数据流交换模型 TFLMS 中加入了代价评估模块。应用融合操作策略优化计算图，使用前向和后向两种搜索方法实现交换策略，弥补了 TFLMS 模型的性能缺陷。

本文通过 Cuda 编程模型和 TensorFlow 框架对访存性能评估方法和数据流交换方法做了实验验证，通过实验发现：本文的 LogGp 数据传输模型可以很好的评估实际数据传输情况；Roofline 模型和预运行分析得到了理论计算峰值和实际运行结果，量化了深度学习算法的计算过程；基于代价的数据流交换方法既利用了 TFLMS 框架扩大系统内存的优势，也保障了访存性能，在原有框架的基础上平均性能提升 4% 左右。

关键字：GPU 微基准测试，访存优化，性能评估

Abstract

In the era of rapid development of artificial intelligence, the application of deep learning algorithms is becoming more and more accurate and performance requirements of deep learning models are more higher. For training deepening neural networks and multi-featured big data, the accuracy of the algorithm is sacrificed while sacrificing the computational performance of the algorithm. So, the performance of the training model becomes the major development limitation. GPU high-speed parallel acceleration devices and acceleration libraries, such as CuDnn optimize deep learning performance, as the implement of many optimisation methods, how to use the GPU's limited memory space to achieve its efficient is a big challenge.

Based on the research of GPU memory architecture, considering the data transfer consumption, this thesis proposes a tensor exchange algorithm to optimize GPU memory performance for convolution-based neural network model. The main works of this thesis are as follows:

The programming methods and GPU memory parameters of the optimized program memory access are obtained, including cache feature parameters, cache strategy, global memory throughput and memory access latency at all levels. This thesis combines the fine grained method with the assembly syntax benchmarking method, which can more fully discover the memory usage limitation factors at all levels, and provide important characteristic parameters for the next data transfer model.

A method for evaluating the performance of GPU microarchitecture for deep learning is proposed. Based on the memory characteristic parameters, data transfer model and computation consumption model on GPU is studied, including the peak estimation model based-Roofline, the data transfer model between GPU-CPU based on LogGp and the actual performance by precalculating. Combining with the GPU experimental platform to modify the evaluation model. The deep learning process is quantified from the perspective of theoretical analysis and actual evaluation, which provides a basis for the next step in memory optimization of the TensorFlow framework.

A data stream exchange method based on data operation cost is proposed. Combined with the previous computational cost model research, the cost evaluation module is added to the existing data stream exchange model TFLMS with the support of theoretical model and actual operational performance. The fusion operation strategy is used to optimize the calculation graph, and the forward and backward search methods are used to implement the exchange strategy, which makes up for the performance defects of the TF-LMS model.

In this thesis, the data computation cost model and the data stream exchange algorithm are verified by the Cuda programming model and the TensorFlow framework. It is found in experiments that the data transfer model of this thesis can evaluate the actual situation well; the computational model is used to quantify the deep learning algorithm to obtain the theoretical peak and the actual operation time; the cost-based tensor exchange algorithm in this thesis not only utilizes the TFLMS framework to expand the system memory advantage, but also guarantees the memory access performance, the average performance 4.6% better than the original framework.

Key words: GPU micro-benchmarking, Memory optimization, Performance evaluation

目 录

第 1 章 绪论	1
1.1 研究背景和意义	1
1.2 国内外研究现状	2
1.2.1 深度学习框架内存优化	2
1.2.2 GPU 运算代价模型	3
1.2.3 GPU 架构微基准测试	3
1.3 研究内容	4
1.4 论文组织结构	5
第 2 章 GPU 访存研究路线分析	7
2.1 GPU 内存微架构探测	7
2.1.1 Set-Associative 缓存模型	7
2.1.2 P-chase 微基准测试	8
2.2 CPU/GPU 运算代价建模分析	9
2.2.1 Roofline 模型	9
2.2.2 LogGp 模型	10
2.3 TensorFlow 深度学习框架	11
2.3.1 TensorFlow 计算模型	11
2.3.2 TFLMS 框架分析	12
2.4 本章小结	13
第 3 章 深度学习平台访存性能	14
3.1 GPU 访存架构分析	14
3.2 GPU 内存架构探测	15
3.2.1 Benchmark 方法	15
3.2.2 缓存架构探测	19
3.2.3 全局内存性能探测	20
3.2.4 共享内存性能探测	23
3.3 基于 LogGp 的数据传输模型	25

3.3.1 模型改进	25
3.3.2 函数性能分析	26
3.4 深度学习算法性能评估	28
3.4.1 基于 Roofline 模型的性能分析	28
3.4.2 预运行时间分析	29
3.5 本章小结	31
第 4 章 深度学习框架访存优化	32
4.1 基于代价的数据流内存交换模型	32
4.2 数据流计算图建模	35
4.3 重写计算图	38
4.4 数据流内存优化模型改进	40
4.5 本章小结	44
第 5 章 实验验证与结果分析	45
5.1 实验方案	45
5.2 数据分析	46
5.2.1 数据传输评价模型数据分析	46
5.2.2 深度学习算法性能评价数据分析	48
5.2.3 数据流内存交换方法数据分析	51
5.3 本章小结	53
结 论	56
参考文献	58
攻读硕士学位期间发表的论文和取得的科研成果	64
致 谢	66

第1章 绪论

1.1 研究背景和意义

伴随着人工智能的飞速发展，深度神经网络的学习方法已经渗透到科学研究的各个领域，正是得益于大量的样本数据、大规模的参数和深层的网络结构对智能化学习精度的保障，在计算机视觉、语音识别、自然语言理解甚至生物医学研究中，都取得突破性的成果。但是，训练神经网络非常的费时。Nvidia 不断的推出高性能计算芯片 GPU，并且针对深度学习做了硬件架构的优化，提供了高达 11.3TFlops 的计算峰值。计算性能和访存性能是深度学习加速的两个出发点，前者已经有很好的硬件平台优化，然而，访存性能是无法在有效地内存空间上有很大提升的。在文献[1]到文献[5]等诸多研究中都证明了访存是如今深度学习模型的性能瓶颈。

如果要发挥 GPU 强大的计算性能，需要编程人员非常熟悉底层架构，并且设计高效的内存使用策略来加速自己的应用，并不能将全部的研究重心放到应用自身的学习精度上。所以为了能够更广泛更方便的使用深度学习算法，涌现出了很多加速库和开源框架，例如 Torch [6], Theano [7], Caffe [8], TensorFlow[20]等，虽然借助框架可以很快上手使用 GPU 编程，但是 GPU 上的应用是对底层架构非常敏感的，需要不断的研究新一代 GPU 架构给开发者提供参考。

在计算机视觉应用中强大的深度学习算法有很多，其中 ResNet [9]已发展到 1001 个神经元结构，而神经网络机器翻译模型^[10]（NMT）由 8 层组成，其中的大多数层是顺序展开的，不可忽视其庞大的内存消耗。所以，有限的 GPU 设备内存与增长的模型复杂性之间的制约，是当前需要研究的一个热点问题。

在之前的学习工作中，针对 Cuda 矩阵乘运算的性能和优化做了大量的测评与分析，研究的主要思路是合理的调度计算和访存指令，使用计算的延迟来掩盖访存的延迟，尽量减少由于访存时延带来的计算单元等待的情况。通过研究发现，高级语言的优化方案能力有限，由于高并发的特点，在经过编译之后，原有的优化方案并不一定能够在底层硬件中发挥作用，通过对已有的最优的矩阵乘算法的深度调试，发现系统的内存吞吐量，活跃的 Warp 单元，缓存的命中率这些性能指标的实际值，与官方测试的峰值相差很大，最高只能达到峰值的 80%。所以，本文在之前研究的基础上，从访存的角度出发，深入底层来探测 GPU 的访存架构，在内存特征参数的基础上，建立评估模型，对上层框架做出优化。

本文对 Pascal 架构内存特征参数的探测与分析，为上层应用提供了基础，同时也为评

估深度学习模型性能提供了重要参考标准。其次，本文对于数据运算和传输的建模，进一步量化了深度学习算法的计算过程，为算法的优化和更深一步了解算法结构奠定了基础，本文的建模方法具有一般性，可以用于其他深度神经网络的研究中。本文的基于运算代价的数据流交换方法，与当前研究中算法级别的优化和矩阵乘计算库的优化相比，效果更加明显，同时也是对 TFLMS 框架的进一步改进，为 GPU/CPU 计算模型框架级别的优化提供了一个新的研究方向。

1.2 国内外研究现状

在面向深度学习的神经网络算法中，内存优化方法大多都是在框架级别提出并实现的，因此，在本节中，首先介绍了现有的国内外现有的访存优化方法，从中发现数据流换入还出方法还有待改进的空间；然后，考虑实现数据流交换的前提条件，介绍了当前国内外对 GPU/CPU 运算代价评估方法的研究现状；最后，介绍了获取 GPU/CPU 运算代价模型中的 GPU 平台重要参数的方法。

1.2.1 深度学习框架内存优化

能够高效地训练大规模模型最有效的方法是使用统一内存^[12]，该方法在 CPU 和 GPU 中访问统一的内存地址空间。启用统一内存的方法很简单，但是与手动卸载和预取数据这种自定义的优化方法相比，它的性能非常差。Shirahata^[13]等人提出在后向运算阶段重复存储区域的方法来提升性能。Rhu^[14]等人提出了一种运行时内存管理方法，在 GPU 和 CPU 上使用虚拟化内存管理策略。在训练期间，只有当前层处于活动状态并消耗 GPU 内存而其他层的数据被换出到 CPU 内存。这种方法的性能与统一内存的方案相比，有很明显的优势。Chen^[15]等人采取了同样的做法通过从 GPU 交换张量来实现 TensorFlow^[16]的内存换出到 CPU 内存，然而，作者并没有说明如何换入换出。并且，通过查阅相关资料，并没有找到 TensorFlow 中的相关实现方案。Tung^[17]等人在文章中首次详细的说明了 TensorFlow 的重写计算图过程，实现了换入换出方法，但是，他们的目的在于增加深度学习模型可训练的图片数量，并没有考虑到数据传输的代价，通过仔细分析他们的实验数据发现，在扩大训练数据规模的同时造成了模型性能的明显下降。除了使用 CPU 内存之外 Chen^[18]等人考虑计算时的临时内存提出梯度检查点的方法，其中检查点使用计算图中的顶点自动定义图分区，检查点顶点之间的图形部分在后向阶段重新计算。所以，在这个优化模型中，前向阶段通常被计算两次。王^[19]等人结合重新计算和换出内存两种方法在单一模型中实现了内存优化。

从上述研究中发现, 数据交换的方法被很多研究者借鉴, 但是并没有定义详细的交换策略, 本文借鉴了文献[19]中的想法, 正式定义了换出换入规则并用于图形重写, 以保证计算图的正确性, 并且, 本文的优化模块可以很好的集成到 TensorFlow 中, 可做到深度学习算法优化的通用性。

1.2.2 GPU 运算代价模型

为了更好的实现本文的优化方法, 需要以 GPU/CPU 计算代价的评估为重要前提条件。在已有的研究中有很多用于建模 GPU 内核执行时间的性能模型。例如, Zhang 和 Owens^[20]开发了一种半经验模型来分析和预测内核执行时间。Hong 和 Kim^[21]以及 Baghsorkhi^[22]等人提出了详细的分析模型来预测 GPU 内核的性能。这些模型旨在通过优化编译器来优化内核, 它们需要有关内核的底层信息, 例如 Warp 和指令级并行度, 这使得它们很难被程序员直接使用。Roofline^[23]模型是一个富有洞察力的高级模型, 它提供了多核架构上并行性能的峰值计算方法。该模型将机器抽象为峰值内存带宽和峰值计算性能两个性能指标。应用程序内核被抽象为单个值(操作强度), 该值描述每个 Dram 流量字节的浮点运算数。使用内存带宽峰值和内核操作强度的乘积作为计算性能峰值。对于 GPU 应用程序, 可以使用该模型, 假设所有数据通过设备的 Dram, 存在缓存缺失或数据重用, 可以使用算术强度代替操作强度。文献[25]中以傅里叶变换运算为例, 指出在以后工作负载中节点间的通信是主要瓶颈。CPU 与 GPU 之间的数据传输可以通过多种方式进行, 文献[26]分析了两种方式对性能的影响: 一种是异步传输; 另一种是将数据块分成多个传输流使得数据传输和运算重叠进行。文献[27]提出 LogGp 模型, 对 CPU 和 GPU 的数据传输进行研究, 分别对显式传输、隐式传输、流传输以及混合传输 4 种传输方式建立数学模型, 并指出当采用某种方式时, 其对应的运算时间和传输时间之间的关系。

上述文献均没有给出运算时间的估计方法。针对上述研究, 本文使用 LogGp 模型进行 CPU 与 GPU 的数据传输, 对现有模型进行改进, 提出一种新的运算代价模型, 并使用 Tensorboard 工具对 CPU 与 GPU 组成的异构系统的性能进行评估。

1.2.3 GPU 架构微基准测试

为了实现 GPU/CPU 运算代价的建模与访存性能评估, 需要获取 GPU 平台自身的内存性能参数。在现有的国内外研究中, 使用特征描述的方法, 一些研究^{[4][5][30]}找出了内存空间的低内存吞吐量的原因, 在文献[34]中设计了数据映射/存储器管理算法, 旨在提高存储器访问效率。最新的研究中, 李^[35]等人建议使用监控机制来更好地利用 L1 数据缓存以获

得更高的性能。这些研究都为这一领域做出了贡献并激发了本文的工作。

本文总结了相关的 GPU 微基准工作。大多数关于存储器结构和访问延迟的工作都是基于 P-chase 微基准测试，在文献[37]中被引入（以下称为 Saavedra1992）。Saavedra1992 最初是为 CPU 硬件而设计的，并且在各种 CPU 平台上得到了广泛的应用^[38]。Duchateau^[39]等人开发了一种 P-chase 的多线程版本，用于探索多核 CPU 缓存架构。他们利用虚拟共享来快速确定缓存一致性协议块大小。由于 GPU 上的多线程功能与 CPU 不同，无法直接在 GPU 上应用它们的方法。Volkov 和 Demmel^[40]在早期的 GPU 设备中使用 P-chase，例如，Nvidia 8800GTX，具有相对简单的内存层次。Papadopoulos^[41]等应用 Saavedra1992 探索全局内存 TLB 结构。他们还提出了一种新颖的方法（即 Wong2010）来研究其他 GPU 缓存^[42]。Baghsorkhiet^[43]等人应用 Wong2010 对 Fermi GPU 进行基准测试并公布其 L1 / L2 数据缓存结构。Meltzer^[44]等人使用 Saavedra1992 和 Wong2010 来研究 Fermi 架构的 L1 / L2 数据缓存。他们发现 L1 数据缓存没有使用最少最近（LRU）替换策略，这是传统 P-chase 测试的基本前提之一^[42]。Zhang 和 Owens^[45]从带宽的角度定量地计算了全局共享内存吞吐量。

通过对比相关研究发现，文献[46]提出的细粒度微基准测试，揭示了 Saavedra1992 和 Wong2010 都忽略的一些独特特征。通过在最近几代 GPU 上的测试研究，验证了该方法的实用性，并论证了传统的 P-chase，Saavedra1992 和 Wong2010 方法并不适用于所有的 GPU 类型。这种基准测试方法能得到更详细的 GPU 内存参数，因此本文在该方法的基础上做了修改，用于 Pascal 平台的基准测试。

1.3 研究内容

为解决当前国内外相关研究中存在的问题，本文提出了以下研究内容：

(1) 探测 GPU 架构的访存特征。首先，分析新一代 GPU Pascal 架构上程序的数据访存过程。然后，将细粒度和嵌入汇编语法的基准测试方法相结合，在大量的微基准测试中总结优化程序访存的编程方法，并探测 GPU 的内存特征参数，包括缓存特征参数，缓存策略，全局内存吞吐量，各级内存访问时延以及影响因素等内容。以弥补新一代 GPU 内存架构研究的空缺，并为本文下一步数据运算传输模型的建模提供了重要的依据。

(2) 针对当前研究中，深度学习算法访存性能评估不全面的问题，本文研究面向深度学习的 GPU 微架构访存性能评估方法。主要包括基于 Roofline 的运算峰值计算模型和基于 LogGp 的 GPU-CPU 之间数据传输模型，由于实际的运行结果会受到当前系统吞吐量和系统带宽的影响，本文进一步分析 Tensorboard 工具，可视化深度学习算法的运行数据和 Tensor

rFlow 计算图，实现深度学习算法的运算过程的量化，获得实际性能和理论性能，为下一步工作提供依据。

(3) 从框架级优化角度出发，为弥补当前 TFLMS 模型的缺陷，本文研究基于代价的数据流交换方法，在现有的内存换出换回模型 TFLMS 的基础上，结合访存性能评估方法，优化数据流的换出换回策略。主要研究内容包括 TensorFlow 内存交换模型分析，数据流计算图建模，重写计算图策略和数据流交换模型改进，使 TFLMS 模型在扩大训练样本数量的前提下，运算性能有所提升。

1.4 论文组织结构

本文内容共有五章，章节组织如下：

第 1 章为绪论。首先，从深度学习算法的访存瓶颈问题出发，介绍了本文的研究背景，并提出了 GPU 内存微架构研究对优化应用访存的重要性；然后，调查了在深度学习框架内存优化、GPU 运算代价模型和 GPU 微架构内存研究方面的国内外现状，并指出了当前研究中的优势与不足；其次，根据研究现状提出了本文主要的研究内容，最后介绍了本文的行文思路和章节安排。

第 2 章为 GPU 访存路线分析。首先，从底层微架构出发，对用到的缓存模型和基准测试技术做了分析；然后，分析了 CPU/GPU 的计算代价建模技术——Roofline 和 LogGp；最后，介绍了上层应用框架 TensorFlow(TF)的计算模式，并分析了 TFLMS 优化模块的缺陷，为后续章节的实现奠定了基础。

第 3 章为深度学习平台访存性能研究。在本章中按照从微观到宏观的研究思路，首先分析了 GPU 的访存架构，将细粒度的微基准测试方法和内嵌汇编语法的测试方法相结合，探测了 Pascal GPU 内存微架构的特征参数；其次，在内存特征参数的基础上，改进 LogGp 模型，完成了 CPU/GPU 数据传输性能的评估；最后，从深度学习算法的角度出发，应用 Roofline 模型完成了理论计算峰值的评估，并且使用预运行的方式，完成了实际运行性能的预测。

第 4 章为深度学习框架访存优化研究。在第 3 章的基础上，提出了考虑运算代价的数据流交换方法。首先，介绍了基于代价的 TF 数据流内存优化模型的整体思路；然后，对 TF 数据流计算图建模，并介绍了重写计算图后的数据流抽象方法；最后在上一节计算模型的基础上，详细描述了基于代价的数据流内存交换方法的实现策略。

第 5 章为实验验证与结果分析。分别介绍了数据传输评价模型、深度学习算法性能评价模型和基于运算代价的数据流交换方法的实验方案、实验结果和数据分析结论，通过对比验证证明了本文第 3 章和第 4 章模型的正确性和高效性。

第2章 GPU 访存研究路线分析

本章将分析文中用到的相关技术，按照从 GPU/CPU 的底层内存架构到 TensorFlow 上层应用的顺序，首先分析了探索各级内存特征的微基准测试方法，其次，在架构特征参数的基础上，分析了量化运算和数据传输性能的 Roofline 和 LogGp 模型，最后，分析了 TensorFlow 计算模型、TFLMS 框架和 ResNet 神经网络。

2.1 GPU 内存微架构探测

本文中的变量定义：

表 2.1 缓存模型变量定义

符号	描述	符号	描述	符号	描述	符号	描述
C	缓存大小	N	数组长度	a	缓存行数量	k	迭代次数
b	缓存行大小	s	步长	T	缓存集数量	r	缓存缺失率

2.1.1 Set-Associative 缓存模型

典型的微基准程序有 P-chase、Saavedra1992 和 Wong2010，最新的微基准测试方法^[47]在 2016 年提出，这些基准测试的前提都是经典的 Set-Associative 缓存模型和 LRU 替换策略。

在内存架构测试中，假设 GPU 与传统的 CPU 相同，使用经典的 Set-Associative 缓存模型和 LRU 替换策略，高速缓存大小(C)比主内存小得多。数据从主存以缓存行为基本单元加载到缓存。高速缓存行中的字长称为缓存行的大小(b)。对于传统的 LRU Set-Associative 缓存模型，缓存内存被分成 T 个缓存集合，每个都包含 a 个缓存行。图 2.1 显示了一个 12 字 Set-Associative 的缓存实例及其内存映射。所有的研究方法都以以下三个假设为基础：

假设 1 所有缓存集具有相同的大小，并且缓存参数满足 $T * a * b = C$ 。其中有三个已知就能得到一个未知的参数；

假设 2 在内存地址中，标识高速缓存集合位之后紧跟着数据内缓存行位置偏移量；

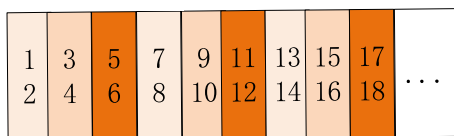
假设 3 缓存替换策略是 LRU。

假设 1 意味着所有缓存集合都有相同缓存行 s 。假设 2 表示数据从主内存映射到高速缓存是有规律并可预见的。例如，在图 2.1 中，每六个连续字被映射到一个缓存集，并且

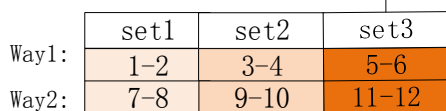
它们可能出现在一个缓存集中的任意一个缓存行中。假设 3 意味着如果本文依次加载一段数据，内存访问是周期性的。以图 2.1 中的缓存模型为例，本文进行了初始化一个包含 13 个字的数组，并一个接一个连续读取。图 2.2 显示了完整的内存访问过程及其访问模式（即缓存未命中或缓存命中情况）。由于数组大小比缓存大一个字，所以出现缓存未命中的情况。在前 12 次数据访问中，这些访问都是冷缓存缺失，这些数据访问第 1，第 7 和第 13 数组元素缓存未命中，其余都是缓存命中。13 至 25 内存访问形成了一个访问模式 P，这个模式重复出现直到数据加载过程结束。这种内存访问模式的周期是 13，刚好为数组的长度。

图 2.1 中是一个 2-way 的 Set-Associative 缓存模型，每一个字有 4 个字节，每一个缓存行存储两个字($b=2$)，数组被顺序访问，所有的缓存行被分到 3 个缓存集中($T=3$)，每一个集有 2 个缓存行，这种模型的缓存联合度为 2($a=2$)以数据单字节地址为例，一个缓存行有两个字，8 个字节，需要三位内存地址来定位缓存行内的数据地址(2 的 3 次方为 8)，一个集地址需要 2 位。

(a) 主存空间



(b) 缓存空间



(c) 内存地址=

set	offset
4	3
2	1
0	0

图 2.1 缓存模型示意图

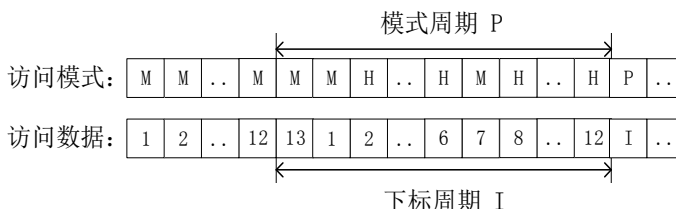


图 2.2 缓存数据访问模式(M-缓存缺失, H-缓存命中)

2.1.2 P-chase 微基准测试

内存参数的特征测试是本文很重要的一部分工作，而所有的基准测试都是在 P-chase 的基础上实现的。所谓的基准测试就是一个获得缓存特征参数的有效方法^[36-38]。核心思想是遍历一个数组并初始化为下一个的索引内存访问。两个连续访问的数组元素的距离被被称为步幅，通常在一个实验中是固定的。由于缓存的影响，内存访问延迟高度依赖步幅。通过测量大量内存访问的平均内存访问延迟，从数组大小和步幅大小可以推断出缓存参数。

基于假设 1-3, P-chase 的输出, 即平均存储器访问延迟 t_{avg} 满足

$$t_{avg} = t_0 * (1-r) + (t_0 + t_m) * r = t_0 + t_m * r \quad (2-1)$$

其中 r 表示高速缓存未命中率, t_0 表示高速缓存存取等待时间, t_m 表示高速缓存未命中访问时间。由于 t_0 和 t_m 是与硬件相关的常量, 因此典型的 P-chase 方法实际上依赖于缓存缺失率 r 。

在 P-chase 的基础上发展而来的微基准测试还有 tvalue-s 图 (Saavedra1992) 或 tvalue-N 图 (Wong2010)。在假设 1-3 下, 存储器访问或缓存缺失模式是周期性的。此外, Saavedra1992 和 Wong2010 都表明, 不仅缓存缺失模式是可预测的, 而且缓存缺失率的可能值也是可预测的。

2.2 CPU/GPU 运算代价建模分析

2.2.1 Roofline 模型

Roofline 模型将浮点性能, 操作强度和内存性能联系在一个二维图中。可以通过硬件规范或微基准测试找到峰值浮点性能。经典的 Roofline 模型^[23]用作内核的操作强度的可视表示, 以反应硬件可实现的最大性能。在模型中, 展示了性能峰值和计算峰值与内存带宽之间的关联。对于较小的操作强度值, 由于缺乏优化内存操作, 内核受系统内存带宽的限制。对于本质上不受内存限制的内核, 通过不断的改进可将它们转换为计算约束的内核。通常, 可以利用 Roofline 模型找到其内核中的瓶颈, 并帮助开发人员找到合适的技术来提高其性能。

在模型中: 计算性能表示为 π , 指的是一个计算平台倾尽全力每秒钟所能完成的浮点运算数, 单位是 FLOp/s。带宽 β , 即计算平台的带宽上限, 指的是一个计算平台倾尽全力每秒所能完成的内存交换量, 单位是 Byte/s。计算强度上限 I_{max} , 两个指标相除即可得到计算平台的计算强度上限。它描述的是在这个计算平台上, 单位内存交换最多用来进行多少次计算。单位是 FLOp/Byte。

$$I_{max} = \pi / \beta \quad (2-2)$$

p 代表模型的理论峰值, Roofline 模型将运算划分出的两个瓶颈区域:

$$p = \begin{cases} \beta \cdot I & I < I_{max} & \text{内存瓶颈} \\ \pi & I \geq I_{max} & \text{计算瓶颈} \end{cases} \quad (2-3)$$

2.2.2 LogGp 模型

本文中使用的是 LogGp 来评估 CPU/GPU 之间数据传输的性能,在文献[29]中详细描述了使用 LogGp 建模的过程,本文中只关注数据传输模型。

(1) 针对 Cuda 流的 LogGp 模型

为了模拟在使用 Cuda 流时可以实现的重叠程度,所有流的总的内核执行时间为 t_E ,传输时间 t_{Thd} (主机到设备)和 t_{Tdh} (设备到主机)表示组合所有流的传输所花费的时间总和。每个流的时间记为 $t/nStreams$ 。由于一些其他的开销,真实的 t_{Thd} 和 t_{Tdh} 通常会大于使用流时的观察时间。因此,LogGp 模型提出的估算属于乐观估计。

针对当前的三种 GPU 传输模式,LogGp 模型提出了不同的计算策略。图 2.4 显示了使用 1,2 和 4 个流的每个类别的计算和通信之间的重叠情况。内核的行为分为两种情况,一种是内核执行时间,是性能的主导因素;另一种是则是 PCIE 传输。左侧的蓝色条表示从主机到设备的数据传输所花费的时间,绿色条表示内核执行时间,一个蓝色条表示将数据从设备传输到主机所花费的时间。从这些图中,可以为使用 Cuda 流的内核的性能推导出简单的分析模型。

第一种,具有隐式同步的设备并有 1 个复制引擎,例如 GTX 480 和 Tesla C2050,还有 Kepler GTX 680。隐式同步有一个操作时延需要做依赖性检查,以查看流内核启动是否完成的操作,直到所有内核从任何流启动的所有线程块都已开始执行。参见图 2.3(a),在内核主导计算过程的场景中,计算可能与从主机转移到设备的流重叠。在传输占主导的场景中,从主机到设备的传输可以与不同流中的计算重叠。由于隐式同步,可以实现计算和从设备到主机的传输之间的重叠。主机到设备传输和计算之间的重叠程度可以如下建模。

$$time_{trans} = t_{Thd} + (t_E / nStreams) + t_{Tdh} \quad (2-4)$$

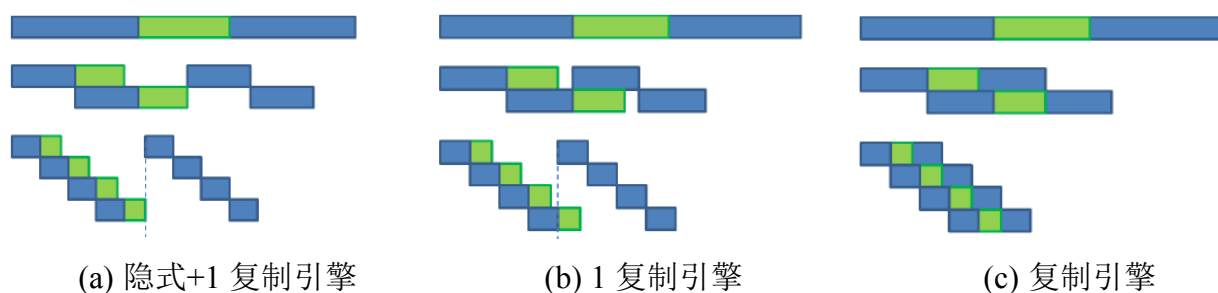


图 2.3 GPU 传输模式

第二种,没有隐式同步并有一个复制引擎的设备,例如 GTX Titan。像 GTX 1080

Ti 这样的设备，计算可以与不同流中任一方向的传输重叠，参见图 2.3(b)。但是，因为只有一个复制引擎，所以相反方向的传输只能在完成第一个方向的所有传输后启动。这意味着内核执行时间可以完全隐藏在数据传输中，具体取决于主机到设备的传输时间是否足够大以隐藏所有计算时间，传输时间建模如下：

$$time_{trans} = \max(t_{Thd} + t_{Tdh}, t_{Thd} + \frac{t_E}{nStreams} + \frac{t_{Tdh}}{nStreams}, t_{Tdh} + \frac{t_E}{nStreams} + \frac{t_{Thd}}{nStreams}) \quad (2-5)$$

第三种，没有隐式同步并有 2 个复制引擎的设备，例如 Tesla K20。使用 2 个复制引擎时，一个方向上的传输可以与不同流中相反方向的传输重叠，请参见图 2.3(c)。这意味着三个时间可以成功地相互重叠。由此产生的执行时间可以建模为最长时间加上其他两个时间中每个流的花费时间。因此，提供单一估计，获得传输计算方案。

$$time_{trans} = \max(t_{Thd} + \frac{t_E}{nStreams} + \frac{t_{Tdh}}{nStreams}, t_E + \frac{t_{Tdh}}{nStreams} + \frac{t_{Thd}}{nStreams}, t_{Tdh} + \frac{t_E}{nStreams} + \frac{t_{Thd}}{nStreams}) \quad (2-6)$$

(2) PCIE 数据传输表示：

主机和存储器之间的通信通过 PCIE 进行。PCIE 是一种基于分组的交换式点对点互连^[48]。模型中只考虑通过 HMA 进行异步传输，这相当于在非默认情况下，在固定内存上使用 cudaMemcpyAsync 流或使用设备映射到主机内存。

LogGp^[50]是在 LogP^[49]和参数化 LogP^[51]性能模型基础上发展而来的。LogP 通过四个参数抽象固定大小的数据的传输：传输等待时间(L)，开销(o)，间隙(g)和处理器数量(p)。延迟是网络中每个字节端到端传播所花费的时间。开销被定义为处理器参与传输或接收的时长。间隙是每个处理器的数据可用的每字节带宽的倒数。LogGp 通过为(G)引入单独的带宽来扩展 LogP 模型。带宽简称数据(g)作为参数保留，以模拟网络的启动瓶颈。本文将使用 LogGp 来评估数据传输时间。

2.3 TensorFlow 深度学习框架

2.3.1 TensorFlow 计算模型

TensorFlow 是一个基于数据流编程(Dataflow Programming)的符号数学系统，被广泛应用于各类机器学习算法的编程实现。其工作流程介绍如下：

- (1) 构建一个计算图，图中的节点可以是 TensorFlow 支持的任何数学操作。
- (2) 初始化变量，将前期定义的变量赋初值。
- (3) 创建一个会话，仅仅构建一个图，这些图不会自动执行计算操作，而是还要显式提交到一个会话去执行，图的执行是滞后的。

(4) 在会话中运行图的计算，把编译通过的合法计算流图传递给会话，这时张量在图节点之间传递进行运算。

(5) 关闭会话，当整个图无需再计算时，则关闭会话以回收系统资源。

TF 上实现的深度学习模型，在执行前向计算的时候，启发式的优化算法通过观察图中的节点的计算顺序，来决定哪种操作放在哪个节点上，从而帮助用户来内存重用；当启发式的算法无效的时候，用户还可以通过添加控制依赖来自行实现内存上的优化。

当反向传播加入的时候，情况变得复杂，在正向计算中较前位置的计算数据在反向传播的后期会被经常用到。这就需要把这些数据存在内存中，从而整个图的内存都将被占用，使得本来就少的 GPU 内存更加的捉襟见肘。有三种方法来对其进行优化：更加复杂的启发式算法来决定图的计算顺序；重新计算这些向量而不是保存下来；将长期在 GPU 内存中的 Tensor 张量转移到 CPU 内存中。在本文中实现了 Tensor 转移到 CPU 内存，并定义了合理的代价模型来评估 Tensor 张量换回的时机。

2.3.2 TFLMS 框架分析

随着网络越来越深，相关研究发现仅仅靠 BN、ReLU、DropOut 等无法解决收敛问题，网络并不是越深越好，一方面因为过多的参数容易导致过拟合；另一方面，训练结果会在真值周围变化，导致网络震荡。所以，训练分类器的时候，用到的 GBDT 和 xgBoost 的思维，借助残差来解决震荡问题。这就是 ResNet 残差网络。ResNet 全称是 Residual Network 是何凯明于 2015 年提出的 CNN 结构模型，该方法以 152 层的网络模型在 ILSVRC 2015 上获得第一名，将错误率降低到了 3.75%。ResNet 的主要优点是可以利用更深层次的网络解决训练误差随网络层数的增加而增大的问题。每一个节点学到的不再是参数本身，而是残差，这就决定了网络有可能无限加深，基线不变，后面的节点学到的是对前面节点的补充，虽然有震荡，但震荡范围越来越小，直到趋于 0。ResNet 的网络结构添加一条从输入到输出的路径，也就是 shortcut 连接，起到非常关键的作用。正是由于 ResNet 网络的以上特点，ResNet 网络更适合本文的实验测试。

TFLMS(TensorFlow Large Model Support)模块^[17]最初的设计理念是通过换出 GPU 上张量的方式增加深度学习神经网络的批处理数量，可以快速的将深度学习模型变为处理大数据的工具。TFLMS 的实现思路是在 TensorFlow 执行之前，使用图编辑模块静态修改计算图，然后将重写的计算图放入到 session 中执行，使用 python 的拓扑定义模块 toposort 来实现计算图的重写过程。本文复现了文献[17]中的实验，得到如图 2.4 所示的实验结果。

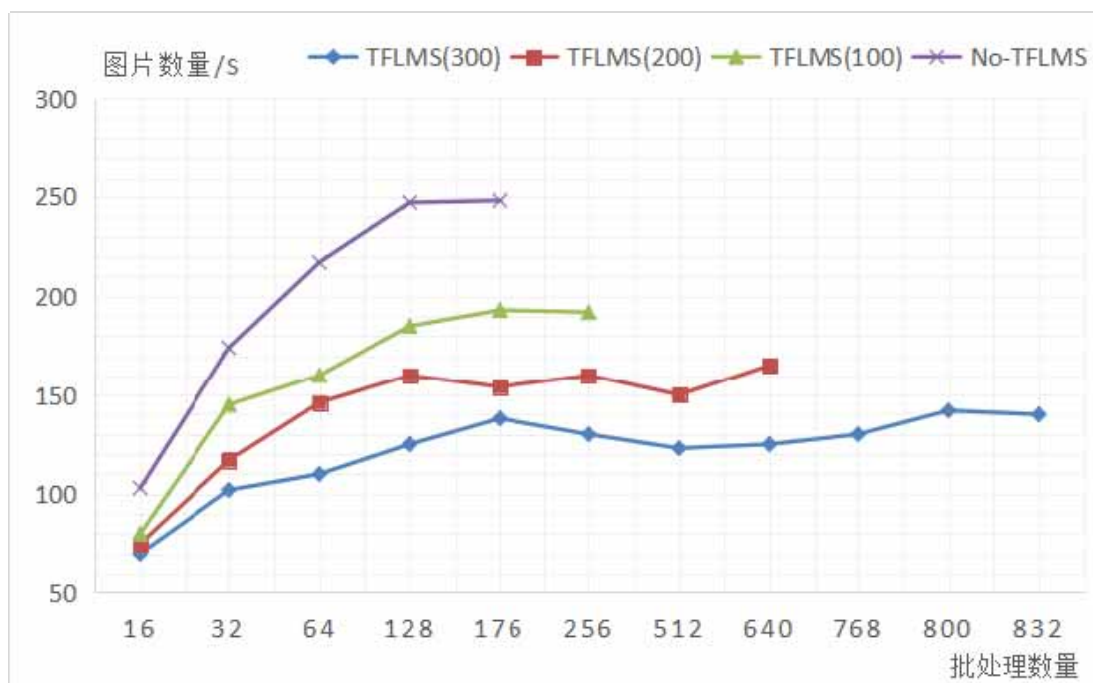


图 2.4 TFLMS 框架测试结果

图 2.4 中，纵坐标表示每秒处理的图片数量，横坐标表示框架处理的批处理数量，从实验结果中可以看出框架扩大了 ResNet 网络的处理数据量，但是，在使用 TFLMS 优化模块的同时，模型的性能下降非常明显，通过进一步分析模块的实现过程发现，数据流换回的策略并没有合理的被定义，所有的数据流都在后向计算的前一个节点换回，当系统吞吐量达到上限时，会出现等待数据的情况，使得系统当前的计算能力显著下降。所以本文从这个问题出发，考虑为深度学习的性能评估建模，应用代价模型量化计算过程，优化数据流交换方法，起到提升框架性能的作用。

2.4 本章小结

本章主要内容是深度学习平台的 GPU 访存优化的研究方法和研究路线，首先分析了缓存架构微基准测试技术，发现经典的 P-chase 测试只是针对传统的 LRU 缓存策略而设计的，对于新型的 GPU 架构并不适用。其次，本章分析了两个运算代价模型，发现 Roofline 模型可以很好的评估计算与内存的瓶颈问题，LogGp 模型需要针对不同的平台做出修改。最后，分析了 TensorFlow 计算模型与 TFLMS 优化框架，发现 TFLMS 框架有数据流换回策略单一并忽略系统性能的缺陷，所以需要进一步优化数据流换回模块来改善性能。

第3章 深度学习平台访存性能

深度学习平台（即 GPU/CPU 计算平台）的访存性能评估，是实现本文优化访存的前提。受到 GPU 内存大小的限制和访存性能的约束，深度学习计算的并行性优势没有最大限度发挥，在诸多研究中发现访存是深度学习计算的瓶颈。所以，在做深度学习神经网络性能优化之前，本节针对使用的 GPU 内存架构做了探测与分析，目的在于发现 GPU 内存的性能参数，在对硬件平台的深入了解的基础上，建立适用于 GPU 的运算和传输模型。最后形成面向深度学习的 GPU 微架构访存性能评估方法。本节的研究内容包括：GPU 访存架构的分析，GPU 内存参数的探测，GPU/CPU 数据传输性能评估，深度学习算法性能评估四个部分。各部分之间的关系如图 3.0 所示。

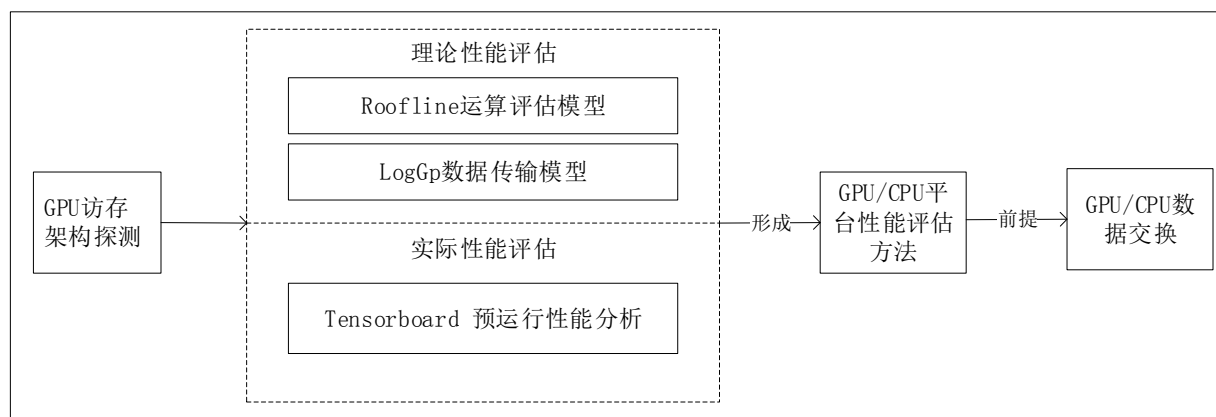


图 3.0 面向深度学习的 GPU 微架构访存性能评估方法架构图

3.1 GPU 访存架构分析

按照 Cuda 语言的定义，GPU 存储空间有六种类型：寄存器，常量存储器，共享存储器，纹理存储器，本地存储器和全局存储器。与文献[42]中研究的早期设备不同，如今 GPU 内存访问都设有缓存机制。本文从 GPU 芯片上和芯片外角度来研究 GPU 内存层次结构与特征。

图 3.1 展示了 Pascal 架构的存储器层次结构框图。箭头表示数据流。与之前架构不同的是 Pascal 架构有单独的共享存储空间，L1 高速缓存与之前的架构也有所改进。在 GPU 程序运行过程中，首先需要将数据从 CPU 拷贝到 GPU 中，CPU 与 GPU 之间的数据传输可以通过多种方式进行，总体上可以分为显式传输、隐式传输、流传输以及混合传输，文献[23]中对这四种传输方式做了详细的分析。根据数据类型的不同，数据被保存到 GPU 上对应的内存空间，全局内存数据被缓存到 L1 缓存空间上，常量数据保存到常量缓存，

显式定义的纹理缓存数据、只读缓存数据保存对应的片上纹理缓存和只读缓存中，当需要计算时，数据从缓存空间读取到寄存器中参与运算。

全局内存和常量内存都占据的是片外存储空间，全局内存使用 L2 高速缓存系统，被所有的流处理器（SM）共享，用于存储指令，数据和访问页表。GPU 访存使用页表的方式将虚拟地址映射到物理地址，通常也存储在全局内存中。TLB 是页表的缓存，一旦线程无法在 TLB 中找到页面条目，它将访问全局内存以搜索页表，这将明显增加访问延迟。

局部内存、共性内存和寄存器存储占据片上内存空间，L1 高速缓存、只读常量缓存和纹理缓存位于每个流处理器（SM）中，在早期的 Fermi 和 Kepler 设备上，共享内存和 L1 数据高速缓存共享存储空间，而在 Maxwell 和 Pascal 设备上共享内存具有专用空间。

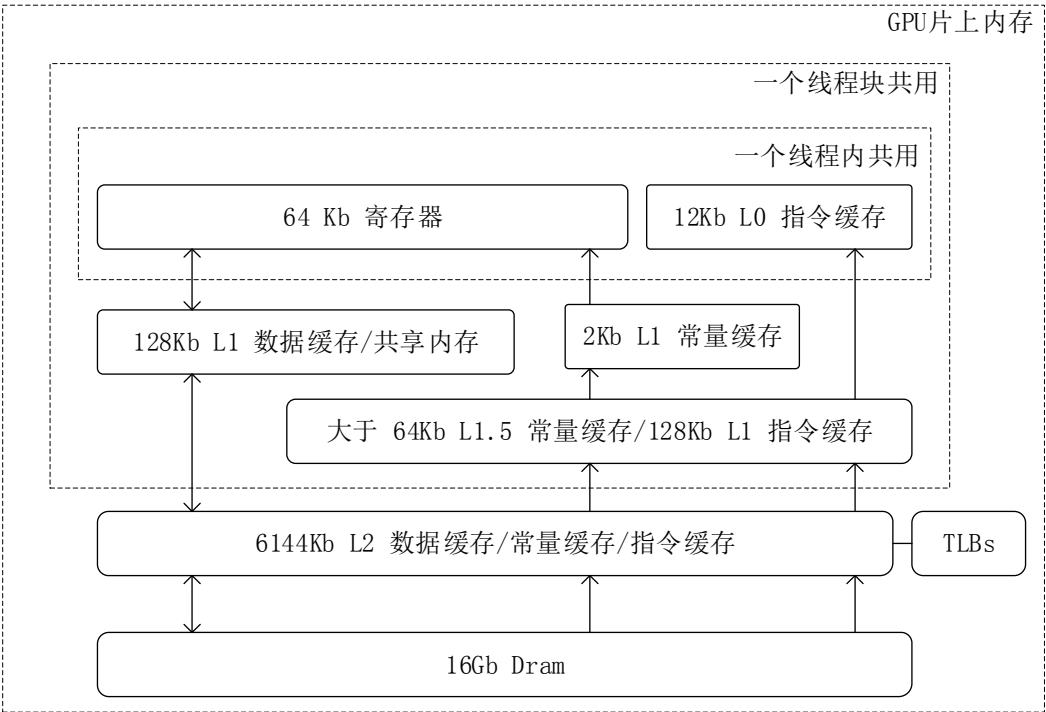


图 3.1 GPU 内存架构图

3.2 GPU 内存架构探测

3.2.1 Benchmark 方法

本节的基准测试将细粒度微基准测试、内嵌汇编语法测试与经典的 P-chase 方法结合，使用细粒度微基准测试对 L1 缓存、全局内存特征做探测，细粒度微基准测试的主要思想是在传统的 P-chase 方法基础上，使用共享内存存储所有数据的访问时间，而不是取多个访问数据的平均值，这种方式可以更好的反映缓存特点，但是并不适用于 L2 缓存的测试，因为 L2 内存空间大，有限的共享内存空间不能存储所有的数据访问信息。所以本文使用

了内嵌汇编语法的测试方式，该方法的主要思想是在高级语言中嵌入汇编语法，为了避免指令级并行，造成的时间测试错误，保证编译过程中不会出现访存指令与时钟获得指令并行发射，同时保证时钟获取指令在编译后位置不被修改，从而更准确的得到数据访问时延。使用经典的 P-chase 方法，通过大量的共享内存空间上的内存复制来完成共享内存访问时延的测试，通过修改线程访问步长完成共享内存 Bank 冲突的分析。

(1) 细粒度微基准测试：

程序 1：细粒度基准测试

```
1 :__global__ void KernelFunction()
2: {
3:  __shared__ unsigned int s_tvalue[];
4:  __shared__ unsigned int s_index[];
5:  predeat the data;
6:
7:  for(it=0;it<iterations;it++)
8:  {
9:    start_time=clock();
10:    j=my_array[it];
11:    s_index[it]=j;
12:    end_time=clock();
13:    s_tvalue[it]=end_time-start_time;
14:  }
15: }
```

细粒度微基准测试程序的核心思想是使用单个线程和单个 CTA（由若干线程组成，硬件最小的执行粒度，相当于 Warp）记录和分析内核中单个数据访问延迟。因为要记录每个数据访问延迟而不干扰正常的数据访问，这种方法难以用于 CPU 高速缓存，但是，可以利用 GPU 共享内存来存储一系列数据访问延迟，共享内存空间独立于其他内存空间，不会影响数据缓存，并且访存性能高，据此本文可以推导出缓存结构和参数。程序 1 给出了本文的单线程细粒度微基准测试的内核代码。在测量之前，需要在初始迭代中访问数据，以便将数据加载到 L2 缓存中。这样做可以避免冷指令缓存未命中以及可能的硬件预取的干扰。第 10 行的核心陈述 `j=my_array[it]`，与传统的 P-chase 相同，区别在于获取时钟的位置，本文把获取时钟语句放在一个长循环中，如第 9 行和第 12 行所示，Cuda 提供的 Clock 函数是通过读取一个特殊的寄存器来实现的，该寄存器的值在每个时钟周期递增。在已有的研究中，将 Clock 语句的开销定义为单个内核线程中两次连续 Clock 调用之间的差值。Clock

的开销为 6 个周期。

在本文中使用同样的方法通过测试发现，Pascal 架构上时钟的开销平均为 4 个周期，本文使用寄存器保存两个时钟周期，因为时钟本身是一种特殊寄存器，如果指定一个寄存器保存，会缩短开销，而且，在寄存器之间计算时间开销可以忽略，所以，本文的测试忽略时钟开销。

细粒度微基准测试需要克服指令级并行（ILP）的问题：函数 Clock 可能与其先前的指令重叠，甚至在前一个指令结束之前返回。例如，如果在语句 $j = \text{my_array}[it]$ （第 10 行）之后立即放置第二个时钟（第 12 行），则可能产生不正确的内存延迟测量，因为第二个时钟可能会在第 8 行完成之前返回。本文通过引入一个新的语句来解决这个问题，即 $s_index[it] = j$ （第 11 行），在第 10 行有数据依赖性，以确保当第 12 行发布时内存访问完成。文献[47]中使用一个单独的程序来测量 10-11 行代码段的开销，在 Fermi, Kepler 和 Maxwell 上分别为 20, 32, 16 个周期。通过本文测试，在 Pascal 架构中该延迟是 10 个周期，进而可以单独推断第 10 行的延迟。

细粒度微基准测试输出的两个数组 $s_tvalue[]$ 和 $s_index[]$ ，其长度等于迭代次数。前者保存数据访问延迟，后者保存访问的数据索引。使用这两个数组，本文可以重现整个内存加载过程，并获取所有数据访问延迟而不是平均值。依据第 2 章中的三个假设，使用具有不同 $(N; s)$ 配置的细粒度微基准程序来完成一个查找缓存参数的执行流程如图 3.2。图 3.2 显示了两阶段的流程图，可以使用蛮力 N 测试来获取缓存大小。

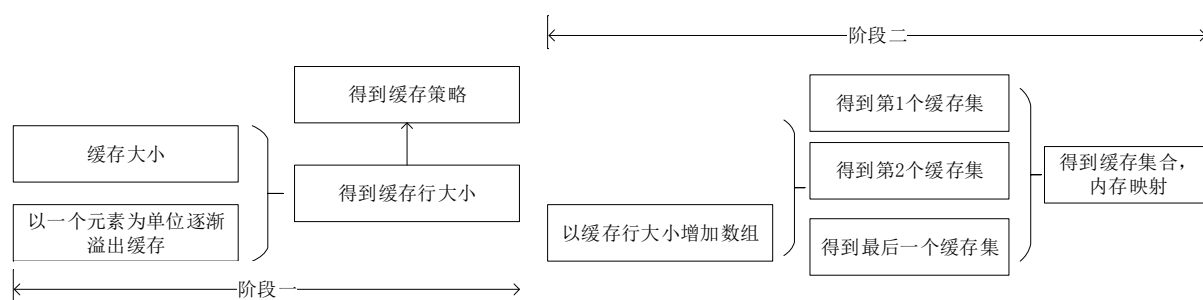


图 3.2 基准测试两个阶段流程

第一阶段，用一个元素溢出缓存，获得缓存行大小，还可以在这个阶段找到缓存替换策略是否是 LRU；第二阶段，逐渐以缓存行的粒度溢出缓存，直到所有数据访问都变成缓存未命中。可以从第二阶段推导出缓存集的数量和内存寻址方式。 $(N; s)$ 的基本单位是数组元素的长度，进一步阐述本文的方法如下：

- 1) 确定缓存大小 C ，设置 s 为 1，然后用一个小的值初始化 N 并逐渐增加，直到出现

第一个缓存未命中， C 等于所有内存访问都是缓存命中的最大 N 。

2) 确定缓存行大小 b 。将 s 设置为 1，从 $N = C + 1$ 开始，然后再逐渐增加 N 。当 $N < C + b + 1$ 时，缓存未命中的数量接近。当 N 增加到 $C + b + 1$ 时，虽然只将 N 增加 1，但缓存未命中数量突然增加。由此可以找到 b 。基于内存访问模式，还可以对高速缓存替换策略有一个总体思路。

3) 确定缓存集合 T 的数量。将 s 设置为 b 。然后，从 $N = C$ 开始，以 b 的粒度增加 N ，每个增量都会导致缓存未命中新的缓存集。当 $N > C + (T - 1)b$ 时，所有的缓存集都被丢失了。可以相应地从缓存缺失模式中推导出 T 。

4) 确定缓存替换策略。如前所述，如果高速缓存替换策略是 LRU，则内存访问过程应该是周期性的，并且高速缓存集中的所有高速缓存路径都会丢失。如果内存访问过程是非周期性的，则替换策略不能是 LRU。在这种情况下，设置 $N = C + b$ ， $s = b$ ，并有相当大的 $k (k \gg N = s)$ ，可以多次遍历数组。所有缓存未命中都来自一个缓存集。每个缓存未命中都是由其以前的缓存替换引起的，因为只通过一个缓存栈溢出缓存。拥有访问过的数据索引，可以重现完整的内存访问过程并查找缓存行如何更新。

应用以上方法，本文简要介绍了 L1 缓存，只读数据缓存。

(2) 内嵌汇编语法测试

程序 2：嵌入汇编语法基准测试方法

```
1 :__global__ void L2_bw(double *dsink,unit32 *posArray)
2: {
3:   UNIT tid=threadidx.x; bid=blockIdx.x;
4:   // 寄存器避免编译优化
5:   double sink=0;
6:   // 从 L2 缓存加载数据，在内核运行之前预热数据
7:   for(i in L2_size)
8:   {
9:     for(j in threads)
10:      UNIT offset=(tid+j);
11:      asm volatile{
12:        ".reg.f64 data"
13:        "ld.global.cg.f64 data"
14:      }}}
```

因为细粒度微基准测试受到共享内存空间大小的限制，而 L2 高速缓存空间中数据量很大，所以细粒度微基准测试不再适用于 L2 缓存架构的测试。本文改进文献[47]中的测试

方法，使用内嵌汇编语法的方法测试 L2 缓存，具体的测试流程如下所示：

为了避免编译过程中的寄存器优化，在第 5 行设置一个双精度变量 sink。该方法包括两层循环控制流，通过外层循环将数据加载到 L2 缓存，内层循环通过嵌入汇编语法的方法先定义一个保存数据的寄存器，然后读取 L2 缓存中的全部数据，在数据加载的前后加入时钟寄存器的读取语句，得到 L2 访问时延。其中“ld.global.cg”汇编表示的是加载数据到 L2 缓存，本文的研究过程中为了不受到 L1 缓存的影响，实验中通过命令行指令的方式来关闭 L1 缓存。

3.2.2 缓存架构探测

(1) L1 缓存

使用细粒度的测试方法发现：在 Pascal 架构中，L1 缓存大小为 24KB，缓存行是 128 bytes，L1 访问时延 84 个周期，L1 缓存测试所得的吞吐量为 31.3 bytes/cycle，理论吞吐量为 128.0 bytes/cycle。

在第一个阶段，发现缓存大小是 24kb；然后设置 s 为 1 个元素（4 个字节长），逐渐溢出缓存，得到缓存行为 32bytes。

在第二个阶段，N 从 24kb 增加到 24.5kb，同时 s=32bytes，直到所有的数据访问都未命中，测试结果如图 3.3 显示。

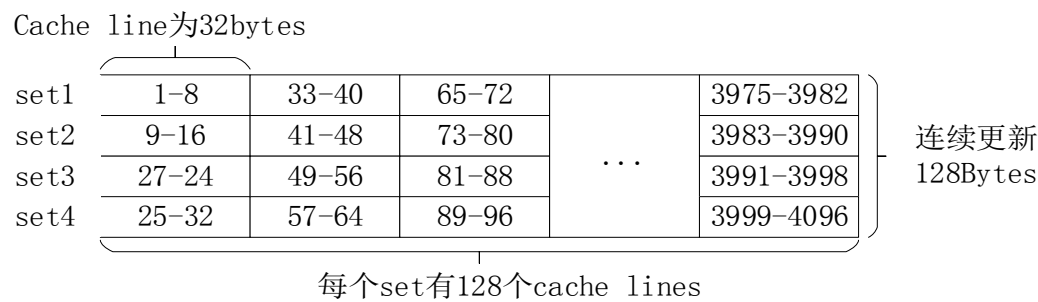


图 3.3 Pascal 架构 L1 缓存架构

如图 3.3 所示，在 Pascal 架构 GPU 上，24kb 的 L1 缓存被分为 4 个缓存集，能存储 768 个缓存行，每一个缓存集包括 128 个缓存行，每个缓存行存储 8 个字（32 字节）。每连续的 32 个字（128 字节）被映射到 4 个缓存集。而且，第 7 到第 8 位内存地址确定了相应的缓存集，然而在传统的 Set-Associative 缓存设计中，第 5 到第 6 位有同样的处理，在图形处理的 2d 空间上，为了利用这种映射的优势，在一般的应用中，同一个 Warp 中的线程需要访问相邻的地址空间，但是一般会造成更多的缓存缺失。

Pascal 架构的缓存策略是 LRU，从实验中发现数据的访存时间呈周期性变化规律。

Pascal 架构改进了 L1 缓存替换策略。当 L1 数据高速缓存饱和时，首先替换相同的四个高速缓存行。这四个缓存行来自不同的缓存集。本文的基准测试表明，Pascal 为 L1 缓存行分配了不同的保留优先级。来自 4 个缓存集的相同的四个缓存行具有最低的保留优先级，首先替换它们，然后是其余的行。与常用的 LRU 策略相比，这种替换策略更适用于保存大型数组，从而不受到稀疏内存访问的影响。

（2）只读缓存

计算能力为 3.5 以上的设备每一个 SM 有一个只读片上数据缓存空间，是 Texture 内存缓存的改进，只读缓存通过调用 `__ldg(const__restricted__*address)` 方法加载，在 Pascal 上，发现有 2kb 的只读数据缓存，使用单精度 4 字节的元组溢出只读缓存，发现缓存行的大小是 64bytes，并且替换策略是 LRU，之后测试 $s=64\text{bytes}$ ， N 从 2kb 变化到 80kb，当数组大于 2.5kb 时，每一个数据访问都缺失。所以推断只读缓存结构为：8 个缓存集，64byte 缓存行大小，每个集中 4 个缓存行，128 个连续的字节映射到 8 个不同的集。

（3）L2 缓存

通过使用内嵌汇编语法的测试方法得到：L2 的缓存大小是 4096Kb，采用 16-way Set-Associative 缓存方式，缓存行的大小是 32B，访问延迟是 234 周期，吞吐量为 1624GB/s。虽然该方法得到了关于 L2 宏观的缓存参数，但是，并不能反应出缓存策略的具体细节，所以本文，试探性的使用细粒度微基准测试对 L2 缓存做了进一步研究。

L2 缓存的缓存策略不是 LRU，因为实验结果显示，内存访问模式不是周期性的；以一个元素为单位逐渐溢出缓存的过程发现 L2 缓存行有 32 字节，试图探测从 Dram 到 L2 缓存硬件级别的预取机制，当使用不规则的 P-chase 步长访问数组元素时，第一个数据的访问时延很长，之后的数据访问时延都符合 L2 缓存的缓存时延，实验中发现如果加载小于 L2 缓存 2/3 大小的数组，将不会发生冷缓存缺失模式。所以，数据预取的大小是 L2 缓存空间的 2/3，并且数据预取是顺序的。

3.2.3 全局内存性能探测

在 Cuda 中，全局内存的访存包括访问 Dram，L1 和 L2 数据缓存，TLBs 和页表，这些都是 GPU 程序中访问非常频繁的内存空间，在本节，通过在 GPU 平台上使用一系列微基准测试来定量的研究全局内存的吞吐量和数据访问时延。

（1）全局内存吞吐量

虽然 GPU 设计为高内存带宽，理论上，带宽的计算公式为：

$$f_{mem} * bus_width * DDR_factor \quad (3-1)$$

f_{mem} 是内存频率，Pascal 架构的内存频率为 5505Mhz，bus width 为 352bit， DDR_factor 是 4，计算的得到理论值为 732Gb/s，但事实上，通过测试发现实际吞吐量最高只有 520Gb/s，只能达到理论峰值的 71%。

全局内存的吞吐量受到许多因素的影响。依据 Little's law，内存访问应该充分利用带宽，本文中执行一个简单地大量的固定的内存复制，测试在 CPU 上全部消失的时间。并计算吞吐量， $2 * datasize / time$ ，在每一组实验中，变化 CTA 数量，CTA 的大小（每一个 CTA 中线程的数量）和 ILP，ILP 定义了每个线程在每次任务中复制 4 字节数据的数量。在实际执行过程中，分配一定数量的 CTAs 给一个 SM，受到了每个 SM 上活跃的线程数的限制，这些 CTAs 并不总是并行执行，每个线程执行数百次数据复制以确保有足够的内存请求，一般的，在 ILP/CTA 值和 CTAs 的数量很大时，吞吐量接近其最大值。本文中发现，吞吐量受到活跃的线程数量的限制，当 CTA 的数量和自身大小都很小时，吞吐量呈线性增长，同时也受到 ILP 的影响，在大 ILP 下的吞吐量很快达到饱和状态，设备依赖于 ILP，因为它的 SM 发射最少的 warps/CTAs，并且一个大的 ILP 可有效地处理更多内存请求的情况。GTX 1080 Ti 有最高的吞吐量，得益于其最高的 bus width，但是它收敛的速度比较慢的，也就是，需要更多的内存访问来掩盖传输的延迟。考虑到在真实的应用中大量并行的内存访问是无法实现的，高 bus width 有一点浪费，这也是 NVIDIA 在 MaxWell 架构中将 bus width 减回到 256bits 的原因。

（2）全局内存访问时延

在本小节中，针对不同的数据访问模式记录了全局内存的访问时延，全局内存访问时延是指访问在 Dram 或者 L2/L1 缓存中的数据的时间，包括页表查找的延迟，本文中使用细粒度微基准测试方法，以保证在一个实验中尽可能的收集更多的内存访问延时，受到 Saavedra1992 方法的启发，本文手动设置数组中的数值为不均匀的访问步长，使用单个 tvalue-s 图表就能显示不同内存访问模式的内存延迟，在测试过程中通过命令行关闭 L1 缓存。图 3.4 展示了传统的 p-chase 方法和本文提出的不统一步长方法的区别。

本文通过命令行指令开启或者关闭 L1 数据缓存。图 3.4 中方块内的数字是数组索引。箭头表示访问数组元素的值，在图 3.4(a)中，数组用单步幅 $s=2$ 访问，形成单一的内存访问模式，只需要加载相邻两个数组元素中的一个，得出的内存延迟也是单一的。在图 3.4(b)中，变换访问步幅 s_1 ， s_2 和 s_3 ，以此获得不同模式下数据的访问时延，统计不同步幅的测试结果得到以下结论：

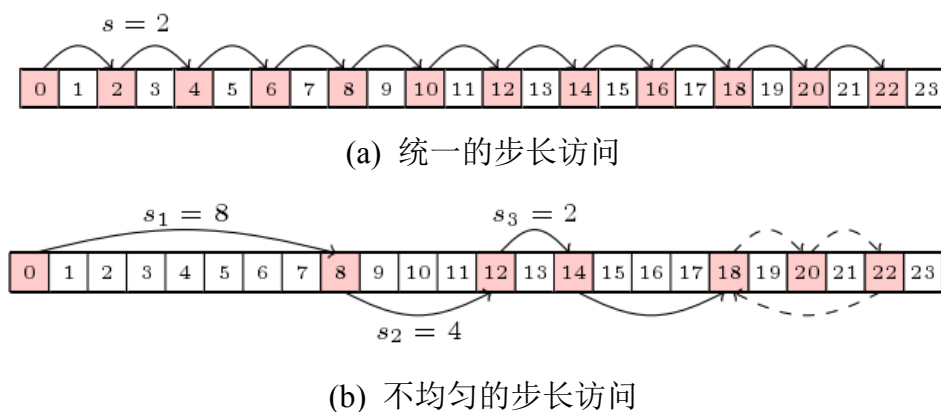


图 3.4 控制步长访问模式

从图 3.5 中可以看出，第一次访问的时延是 1029 个周期，代表是 L2 缓存未命中和 TLB 未命中的结果。对存储在同一 L1 高速缓存行中的数据访问时延很低只有 28 周期，该周期即为 L1 高速缓存命中延迟。193 周期延迟表示 L1 缓存未命中和 L2 缓存命中，375 周期延迟反映了 TLB 命中但 L2 缓存未命中延迟。

在 GPU 架构中 L1 数据缓存通过虚拟地址索引，L2 高速数据缓存通过物理地址索引。因为 L2 是物理高速缓存，所以对它的访问关联到 TLB。通过调整数组大小以超过 L1 和 TLB 覆盖范围，如果 L1 数据高速缓存通过物理地址索引，则基准测试中的访问将导致至少一级 TLB 未命中。在第二次扫描中看不到 TLB 未命中，只要步幅足够大，可以缓存 L1 数据缓存中的所有访问。相同的基准测试表明，当禁用 L1 数据高速缓存时，L2 数据高速缓存中的寻址数据将通过 TLB。在可用的全局内存大小内，GPU 上有两个级别的 TLB。L1TLB 具有 2Mb 页面条目和 32Mb 覆盖范围。L2TLB 的覆盖范围约为 2048Mb。

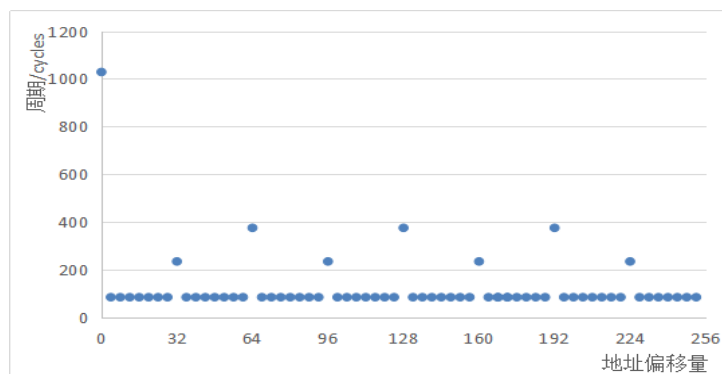


图 3.5 缓存访问延迟测试结果

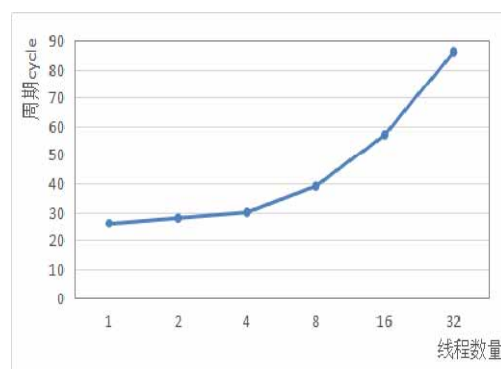


图 3.6 共享内存平均访问时延

3.2.4 共享内存性能探测

(1) 共享内存访问延迟

共享内存访问时延的测试用过大量的共享内存空间上的数据复制来实现，测试程序如下图所示：

程序 3：共享内存探测方法

```
1 : for(i=0;i<=iterations;i++)
2: {
3:   data=threadIdx.x*stride;
4:   //防止冷访问缺失
5:   if(i==1) sum=0;
6:   start_time=clock();
7:   repeat64(data=sdata[data])
8:   end_time=clock();
9:   sum+=(end_time-start_time) ;
10:}
```

本文使用单个 CTA 启动 Warp 线程来访问固定步长的内存。用一个整数 stride 乘线程 i 来获取一个共享内存地址，执行 64 次内存访问并记录总时间消耗。然后计算每次内存访问的平均内存延迟。通过实验发现平均共享内存访问时延随线程数的增加逐渐提高，结果如图 3.6 所示。

图 3.6 统计的是共享内存访问时延的平均情况，如果深入分析造成时延不同的原因，需要进一步研究 Bank 冲突的影响，共享内存空间被分为 32 个 Bank 单元，如果多个线程同时访问一个 Bank 单元，将会产生 Bank 冲突，增加访问时间。当存在 Bank 冲突时，共享内存的访问时延将受到很大的影响，图 3.7(a)显示了通过控制内存访问步长引起的 2 路 Bank 冲突的情况，此时，0 字节和 96 字节同时映射到了一个 Bank 中，如果控制线程步长为 6，那么，线程 0 和线程 16 将分别访问 0 字节和 96 字节，会造成 2 路 Bank 冲突。潜在的 Bank 冲突数量等于访问步长和 32 的最大公因数，奇数的步长不会造成 Bank 冲突。

Pascal 架构通过将存储体宽度加倍来避免共享内存冲突，优于早期的架构。Pascal 存储区宽度为 8 个字节，但它为程序员提供了两种可配置模式：4 字节模式和 8 字节模式。在 8 字节模式中，64 个连续的整数被映射到 32 个连续的存储体上，而在 4 个节点中，32 个连续的整数被映射到 32 个连续的存储体上。图 3.7 示出了两种模式的数据映射。

本文通过控制步长测试了存在 Bank 冲突的情况下，4 字节和 8 字节访问模式的共享内存访问时延情况，图 3.8 展示了实验结果：

从图 3.8 中发现：当步幅为 2 时，任何一种模式都没有 Bank 冲突；当步幅为 4 时，两种模式都显示出 2 路冲突；当步幅为 6 时（图 3.8），4 字节模式存在 2 路冲突，但 8 字节模式没有冲突。对于 4 字节模式，访问一半共享内存。线程 i 和线程 i+16 访问同一存储体中的数据，对于 8 字节模式，32 个线程访问 32 个不同的 Bank，没有冲突。类似地，如果 8 字节模式的数量不是 2 的幂，则 8 字节模式优于 4 字节模式。

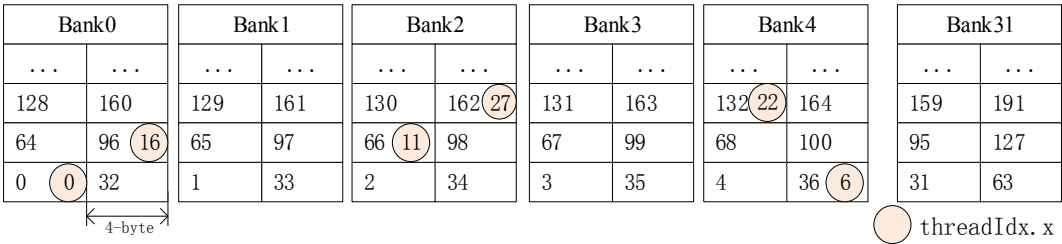


图 3.7(a) 4-byte 模式两路 Bank 冲突

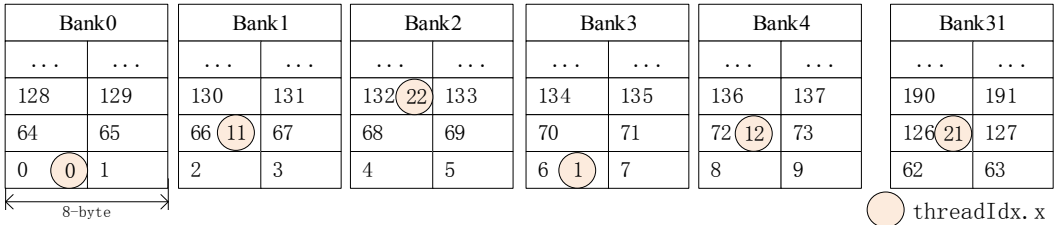


图 3.7(b) 8-byte 模式没有 Bank 冲突

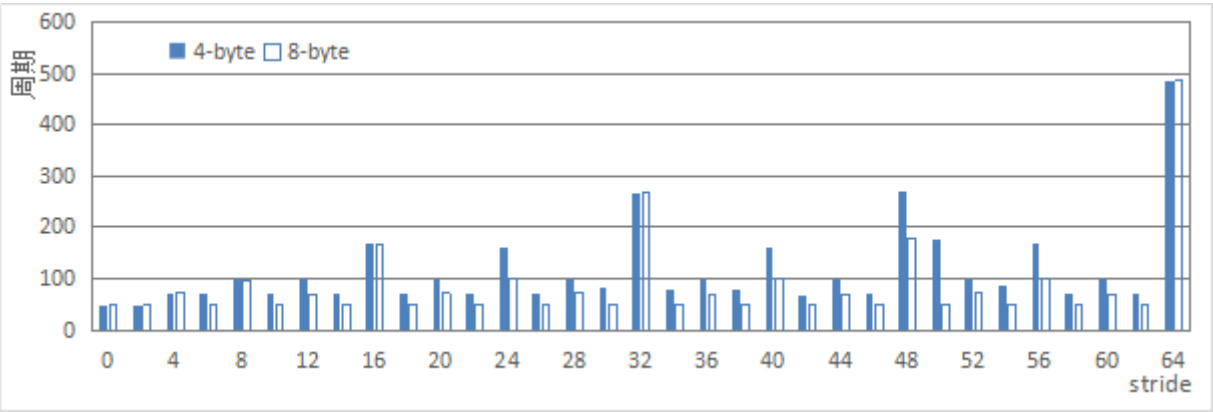


图 3.8 不同步长下访问周期

本文根据潜在的 Bank 冲突数列出了共享内存访问延迟如表 3.1。内存访问延迟随 Bank 冲突数量的增加线性增长。进一步明确了在存在 Bank 冲突的情况下数据访问指令会被顺序执行。通过文献[47]了解到，对于 Fermi 和 Kepler 设备，其中存在 32 路存储体冲突，访问共享存储器需要比常规全局存储器（TLB 命中，高速缓存）更长的时间。然而与之不同的是，Bank 冲突对 Pascal 设备上共享内存访问延迟的影响是温和的。最长的的共享内存访问延迟仍然与 L1 数据缓存时延相当。

表 3.1 Bank 冲突影响下共享内存平均访问时延

Bank 冲突	2-way	4-way	8-way	16-way	32-way
GTX 1080 Ti 时延	26	30	38	53	82

综上所述，虽然共享内存具有非常短的访问延迟，但是如果存在许多的冲突，其访问时间会相应增加。共享内存被设计为高带宽，低内存访问时延的特征，并且每一个 SM 有一块专用的共享内存空间，在 Cuda 编程中，不同的 CTAs 被分配到同一个 SM 共享一个物理内存空间。在 Fermi 架构和 Kepler 架构中，共享内存和 L1 缓存在物理空间上是整体的，在 Maxwell 平台上，共享内存有独立的空间，在利用 GPU 加速的应用中，将数据存储到共享内存是一种优化策略^[53-55]，编程人员在算法执行的前后，将数据从全局内存中移进或移除共享内存，来避免直接从全局内存访问产生很长的访问时延。

3.3 基于 LogGp 的数据传输模型

3.3.1 模型改进

本小节以上一章基准测试的为基础，在现有的性能评价模型的基础上，考虑了数据传输的评估并以 Pascal 架构为实验平台对模型参数做了修改，为后续的内存优化算法提供了理论支持。在 LogGp 模型中，为了模拟通过 PCIE 总线的主机和设备之间的内存传输性能，传输 k 个字节的单个数据流需要时间为 $(L + o + (k - 1)G + o)$ 。其中，传输开启延迟表示为 L ，第一个 o 表示头字节额外，第二个 o 用于表示处理器接收操作花费的时间， G 表示一个大消息的带宽。本文将模型调整为： $(L + o + kG)$ 。LogGp 中的第二个 o 用于表示处理器接收操作花费的时间。在本文的例子中，传输通过 HMA 进行，数据直接写入接收端的存储器，因此没有接收处理器开销。本文的 o 表示发送开销，可以看作控制器注册 HMA 请求的时间。本文用 k 取代 $(k - 1)$ ，它只是 LogGp 模型的一部分，在 $k=1$ 时模型变为 LogP。因为所有传输都是通过 HMA 发生的，所以发送第一个字节不被视为发送处理器开销的一部分。不包括 (-1) 显得更合理。

在 LogGp 模型中发送大量数据被建模为 $(L + o + (k_1 - 1)G + g + (k_2 - 1)G)$ ，其中 g 表示再次传输的等待时间。正如在 LogP 和 LogGp 一样，本文假设单个端口模型，即在任何给定时间只有一个传输可以活动。因此，如果使用多个流来传输数据。假设每个流传输一个接一个地发生。因此，使用以下公式模拟数据的传输时间。例如，模拟 2 个流，每个传输一半的字节被建模为

$$t = L + o + \frac{1}{2}kG + g + \frac{1}{2}kG \tag{3-2}$$

或更一般地，对于 nStreams 流，评估公式定义为

$$t = L + o + \frac{k}{nStreams}G * nStreams + g * (nStreams - 1) \tag{3-3}$$

其中，对于同一个设备，参数是固定的，在本文使用的 Pascal 架构 GPU 上，模型计算的参数如下表所示：

表 3.2 CPU 到 GPU 模型参数

CPU 到 GPU					
$L + o$	0.009420ms	G_{hd}	8.318392E-8ms/byte	g_{hd}	0.002503ms
GPU 到 CPU					
$L + o$	0.009023ms	G_{dh}	7.924734E-8ms/byte	g_{dh}	0.002674ms

通过以上方法可以完成理论上的数据传输时间评估，在 CPU 到 GPU，和 GPU 到 CPU 传输方向的数据传输性能，需要带入数据分别计算。如果是在运行上层应用的情况下，高级语言对数据的传输方案做了不同的封装，为了更准确的把握不同大小的数据传输性能，本文还分析了 Cuda 语言封装的不同数据传输方法的性能。

3.3.2 函数性能分析

分配 CPU 要访问的内存通常使用 C 标准库函数 malloc 来完成。另一种方法是使用 Cuda 的 cudaMallocHost 函数，该函数提供页锁定内存，可在 CPU 和 GPU 内存之间提供更高的传输吞吐量。使用 cudaMallocHost 分配的内存的吞吐量在 Barracuda10 的基础增加约为 2.4 倍，在 Barracuda04 上为 2.0 倍，在 Barracuda01 上为 1.5 倍。这两个方法的区别还表现在于它们是否使用固定内存。固定内存是使用 cudaMallocHost 函数分配的内存，它可以通过换出来防止内存溢出，并提供更高的传输速度。非固定内存是使用 malloc 函数分配的内存。

分配 GPU 要访问的内存通常使用 Cuda 函数 cudaMalloc 来完成。还有其它方法，例如 cudaMallocPitch 和 cudaMallocArray，但本文没有探讨它们，因为 cudaMalloc 性能更高。

图 3.9 是对三种数据复制方法的性能测试，通过实验发现使用 cudaMallocHost 分配内存比 malloc 消耗时间更多。每次调用 cudaMallocHost 的时间保持恒定，大约 2,300 微秒，直到数据量大于 1Mb，调用时间开始增加。在内存大小为 512Mb 时，cudaMallocHost 需要不到 6 毫秒。总的来说，cudaMallocHost 比 malloc 慢三到五个数量级。对于 cudaMalloc 函

数的分配时间消耗情况：当数据量小于 256 字节时，每次调用的时间恒定，略小于 1 微秒。在 2Kb 到 4Kb 之间时间显著增加，直到 512Kb 时间开销都相对恒定为 50 微秒。高于 512 Kb，开销再次增加，在 512Mb 时高达 12.5 毫秒。总的来说，对于小于 4 MB 的内存，cudaMalloc 比 malloc 慢一个半数量级。超过 4Mb 时，cudaMalloc 比 malloc 慢两到四个数量级。

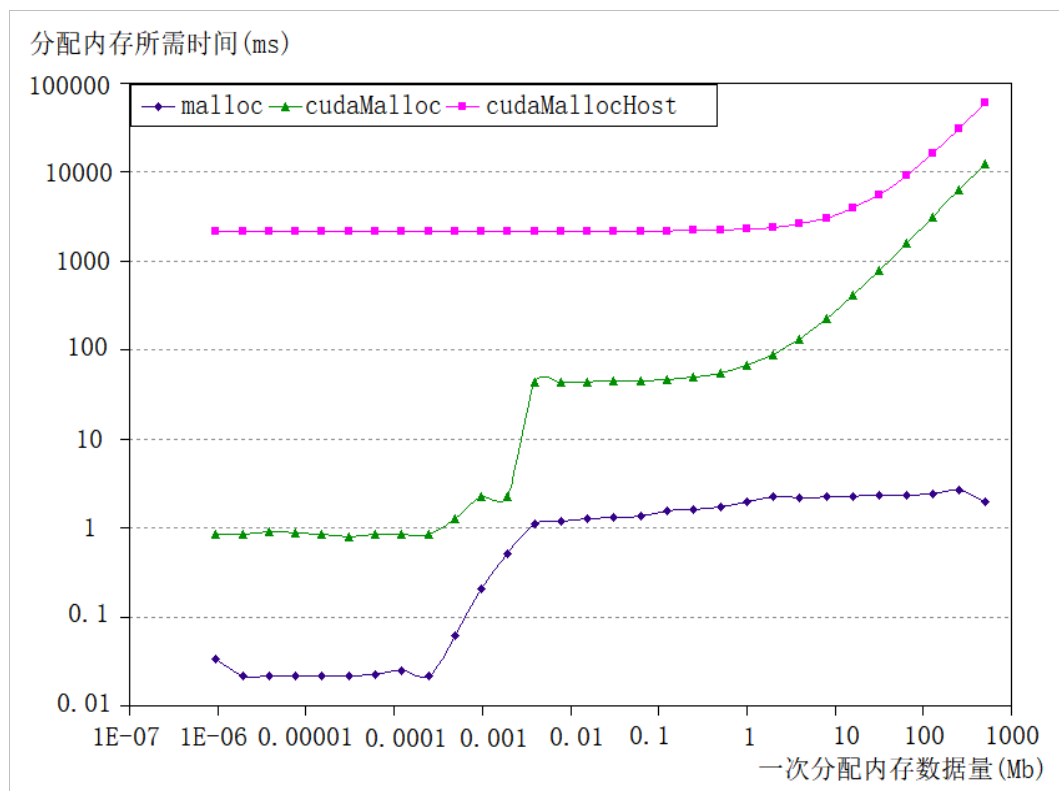


图 3.9 数据复制方法性能测试结果

为了挖掘更多地数据传输性能特征，本文还统计了平均为每个字节分配内存所需的时间，单位是 ns，结果如图 3.10 所示，从图中可以发现，最高的数量级是 10 的 7 次方数量级，而最低的有 10 的 -6 次数量级，差距很大，横纵坐标通过单位转换然后相乘可以得到总的传输时间，以 cudaMallocHost 函数的内存分配数据为例，最高的传输 1000Mb 的字节，所需时间为 0.01ms，最低传输 0.0000001MB 的字节，传输时间为 0.001ms，虽然数据量相差 9 个数量级，但是数据传输的时间差距只有一个数量级，在 Cuda 语言的内存分配函数性能测评实验中，得到以下结论：在三种数据内存分配方式中，cudaMallocHost 分配方式的启动最耗时，cudaMalloc 次之，malloc 用时最短；三种方式都是对于数据量不敏感的，随着数据量的增加，消耗的时间体现出很平稳或微小的上升趋势。

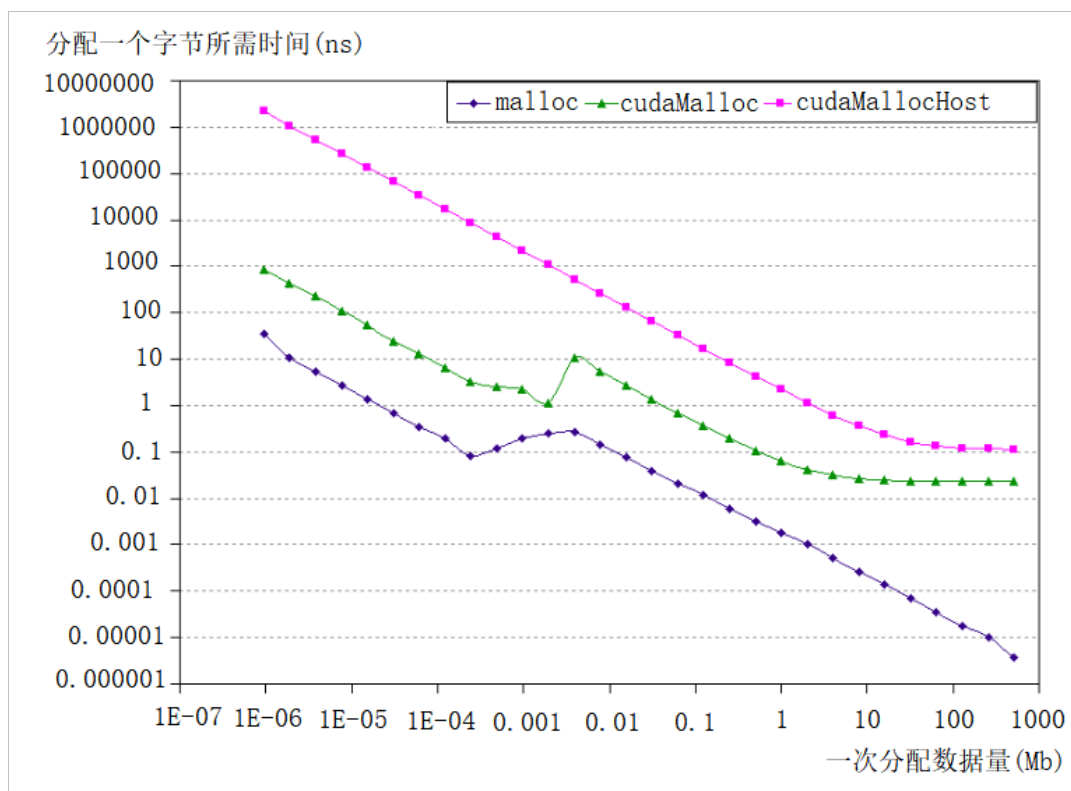


图 3.10 数据内存分配性能测试结果

3.4 深度学习算法性能评估

3.4.1 基于 Roofline 模型的性能分析

在本小节中将通过对 ResNet-50 模型的静态分析，使用 Roofline 模型获得模型的理论计算峰值。为后续的数据换入换出代价的建模和内存优化模型的效率提供了重要的参考标准。使用 Roofline 模型对 ResNet 网络建模：

单个卷积层的计算量：指的是输入单个样本（一张图像），模型进行一次完整的前向传播所发生的浮点运算个数，也即模型的时间复杂度。单位是 FLOps。其中卷积层的计算量公式如下

$$M^2 K^2 C_{in} C_{out} \text{ (FLOps)} \quad (3-4)$$

其中， M 代表每个卷积核输出特征图的边长， K 表示每个卷积核的边长， C_{in} 每个卷积核的通道数，也即输入通道数，也即上一层的输出通道数， C_{out} 本卷积层具有的卷积核个数，也即输出通道数。对于有 D 层卷积的模型整体的计算量表示为：

$$\sum_{l=1}^D M_l^2 K_l^2 C_{l-1} C_l \text{ (FLOps)} \quad (3-5)$$

输入单个样本的访存量，是指模型完成一次前向传播过程中所发生的内存交换总量，也即模型的空间复杂度。在理想情况下（即不考虑片上缓存），模型的访存量就是模型各层权重参数的内存占用与每层所输出的特征图的内存占用之和。单位是 Bytes。由于数据类型通常为 float32，因此需要乘以四。

$$4(M^2 C_{in} C_{out} + K^2 C_{out}) \text{ (Bytes)} \quad (3-6)$$

与计算量类似，可以推导出模型整体的访存量表示如下：

$$4\left(\sum_{l=1}^D M_l^2 C_{l-1} C_l + \sum_{l=1}^D K_l^2 C_l\right) \text{ (Bytes)} \quad (3-7)$$

1080Ti 的运算能力为 $p=11.3TFlop/s$ ，带宽为 $\beta=484GB/s$ ，理论运算强度为 $I_{\max}=24$ 。以 ResNet50 为例，本文通过拆解 ResNet 内部的结构，对计算量和访存令做静态分析，得到一下结果：ResNet 的计算量总共为 3.87GFlops，参数量为：25.7Mb，对每一层卷积核的大小统计可以得到每一层运算的输入输出数据量。通过这个数据可以看出，ResNet50 的计算强度远远小于平台 1080Ti 的计算强度，所以 ResNet 网络的性能瓶颈在访存。因此，本文将以 ResNet 网络为实验对象来验证本文提出的内存优化模型的有效性。

下表是本文静态分析 ResNet50 网络模型的计算和访存的统计数据：

表 3.3 ResNet-50 计算量访存量统计

计算层	计算量	参数量
ResUnitA	$5whcn + 13whcc + 10whc$	
ResNetB	$whcn + 13whcc + 6wh$	25,609,152
conv1	118,816,768	9,664
res2a	233,218,048	76,288
res2b/2c	219,570,176	142,336
res3a	296,439,808	381,952
res3b/3c/3d	218,968,064	844,800
res4a	295,938,048	1,517,568
res4b/4c/4d/4e/4f	218,667,008	5,601,280
res5a	295,687,168	6,049,792
res5b/5c/5d	218,516,480	8,937,472
fc	2,048,000	2,048,000

3.4.2 预运行时间分析

本文中使用的计算图分析工具—Tensorboard，Tensorboard 和 TensorFlow 程序跑在不同的进程中，Tensorboard 会自动读取最新的 TensorFlow 日志文件，并呈现当前程序运行的

最新状态，通过使用这个工具可以很直观的看到整个神经网络的结构，同时也可以展开看每个 layer 中的具体的结构。

Tensorboard 不仅可以展示计算图的结构,还可以展示 TensorFlow 计算图上每个节点的基本信息以及运行时消耗的时间和空间。这可以帮助更加有针对性地进行 TensorFlow 程序,使得整个程序的运行速度更快。使用 Tensorboard 可以非常直观地展现所有 TensorFlow 计算节点在某一次运行时所消耗的时间和内存。本文将在第五章实验部分统计分析 ResNet 模型的预运行情况。工具的使用流程:添加记录节点: `tf.summary.scalar/image/histogram()`等;汇总记录节点: `merged = tf.summary.merge_all()`;运行汇总节点: `summary = sess.run(merged)`,得到汇总结果;日志书写器实例化: `summary_writer = tf.summary.FileWriter(logdir, graph=sess.graph)`,实例化的同时传入 `graph` 将当前计算图写入日志;调用日志书写器实例对象 `summary_writer` 的 `add_summary(summary, global_step=i)`方法将所有汇总日志写入文件;调用日志书写器实例对象 `summary_writer` 的 `close()`方法写入内存,否则它每隔 120s 写入一次。如图 3.10 所示,为 Tensorboard 工具中最外层计算图表示,图 3.11 为 ResNet 计算节点的张量、计算时间和占用内存信息。

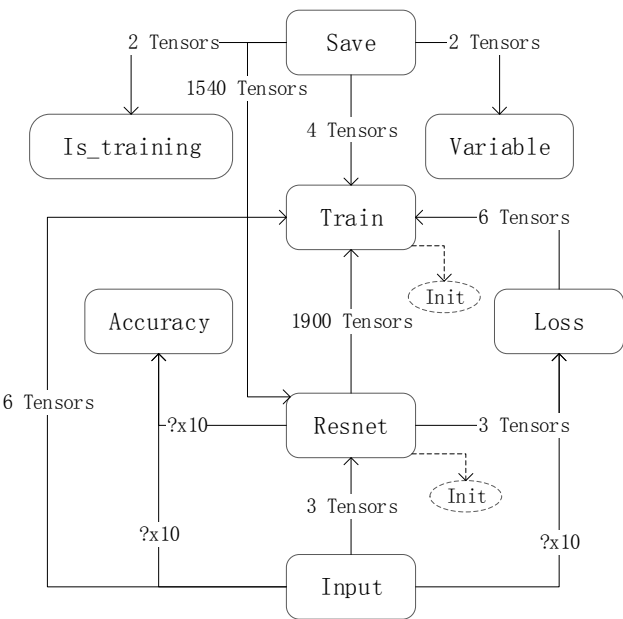


图 3.10 Resnet 网络计算图

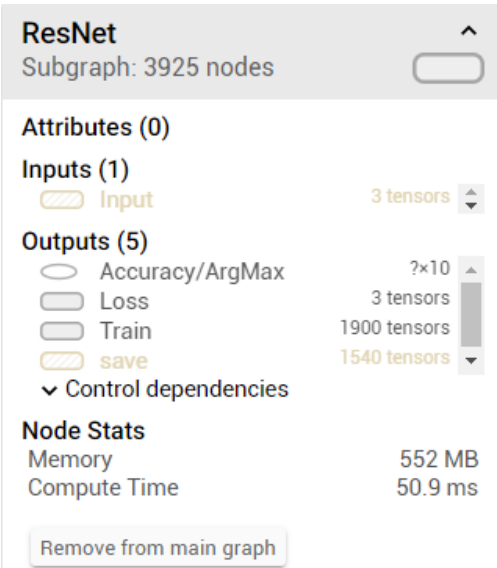


图 3.11 Resnet 节点信息

从 Tensorboard 工具的可视化结果中发现 ResNet 神经网络的计算图结构和输入输出的张量信息,通过查看每个节点的详细信息可以看到每个节点的占用内存,计算时间,和输出到后向计算的张量数量,通过这些指标,结合本章的数据传输模型,可以对深度神经网络的整个计算过程。

3.5 本章小结

本章提出了面向深度学习的 GPU 微架构访存性能评估方法，从理论分析和实际预测两个方面量化深度学习模型的计算过程。首先，将细粒度微基准测试与内嵌汇编语法方法结合，得到了 Pascal 架构的内存特征参数。其次，本章改进了 LogGp 数据传输模型并作了内存函数性能分析，得到了 GPU-CPU 数据传输性能评价方法。最后，本文分析了深度学习算法的理论峰值计算模型，得到了访存量和计算量的权衡指标，并通过 Tensorboard 可视化工具，将深度学习的实际运行过程量化，为性能优化提供了更直观更准确的数据展示平台。

第4章 深度学习框架访存优化

本章中将介绍在上一章内存架构和性能模型的基础上提出的基于代价的数据流内存交换方法，首先介绍优化模型的整体思路，然后，对 TF 的运算模型建模，介绍重写计算图的方法；最后，将基于代价的数据流内存交换方法加入到 TFLMS 框架中，在数据流传输性能和运算性能的限制下，介绍了优化计算图的两策略，并用前向和后向两种搜索方式实现了本文的优化方法。

4.1 基于代价的数据流内存交换模型

本章改进了数据流内存交换模型，在模型中加入了代价评价模块，模型的整体思路如图 4.1 所示。

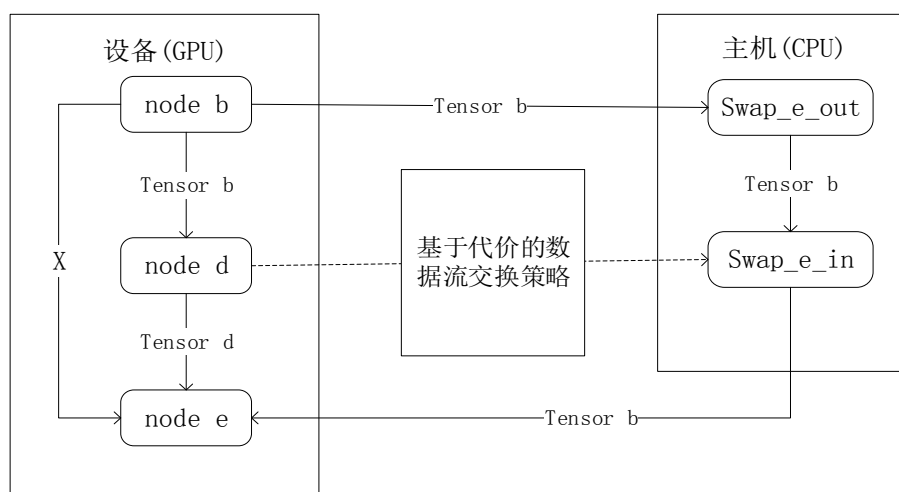


图 4.1 数据流优化模型整体框架

模型中共有三个关键部分构成，包括 TensorFlow 上数据流计算图、依赖控制策略和 CPU/GPU 数据传输。其中数据传输模型和计算图中节点计算评估方法在第 3 章中已经做过研究，本节将重点介绍 TensorFlow 上的计算图和模型中的依赖控制部分。图 4.1 表示本文优化框架的主要思路：通过切断特征映射和后向节点之间的参考边来重写数据流图，同时在相关节点中插入交换算法，以保证逻辑等价。这些插入的节点将放置在“CPU”设备上。因此，在实际执行之前，将在张量流中透明的插入跨越设备的 CPU 张量通信节点。如图 4.1 所示，张量 b 在传送到 CPU 存储器后可立即释放。当它被节点 e 重用，它可以被传送回 GPU 内存。

首先，将分析计算图中的数据流。**TensorFlow** 使用统一的数据流图来表示模型计算任务。如图 4.2 所示，节点（Relu_fwd 等）表示计算。边（Relu 等）在节点之间传递张量（表示数据或依赖）。每个节点的执行都在计算图执行之前被预定好，可以作为训练任务的中间过程进行查看，因此图上的优化对模型而言是透明的。

TensorFlow 的数据流模型原型是数据流机，在应用广泛的深度学习算法中，连接式的算法往往需要使用梯度下降法来求取参数，TF 通过扩展图的方式实现了自动求导。对于每张计算图，得到从输入到输出的路径，并从输出到输入回溯，回溯过程中对于路径上的每个节点，添加另一个节点来计算偏导的新节点，如图 4.2 中_bwd 后缀表示的节点，在计算偏导的过程中，新节点不仅仅将上一层传下来的反向导数作为输入，还可能将前向计算节点的输入和输出也作为其输入，如图 4.2 中星号标记的边。**TensorFlow** 的内存管理在内存分配过程中使用动态策略。这种策略的重要前提是确定张量分配和解除分配的时间。在程序运行期间，开始执行相应的节点时，张量被分配内存，其引用计数增加，当它的引用计数减少到 0 时，它的解除分配操作将被自动触发。在图 4.2 中，因为 Relu_fwd 的引用计数为 3，所以 Relu 的引用计数为 3。Relu_bwd 完成后，Relu 的引用计数变为 0，然后释放。Relu 的生命周期从 Relu_fwd 开始运行到 Relu_bwd 完成而结束。所以，本文发现在正向阶段中生成的所有特征图都会逐渐积累，直到损失函数层完成，在文献[56]中以 ResNet-50 网络为例，统计了在前向计算阶段产生了最大内存占用的情况，并指出这种特点限制了内存约束下的模型复杂度。

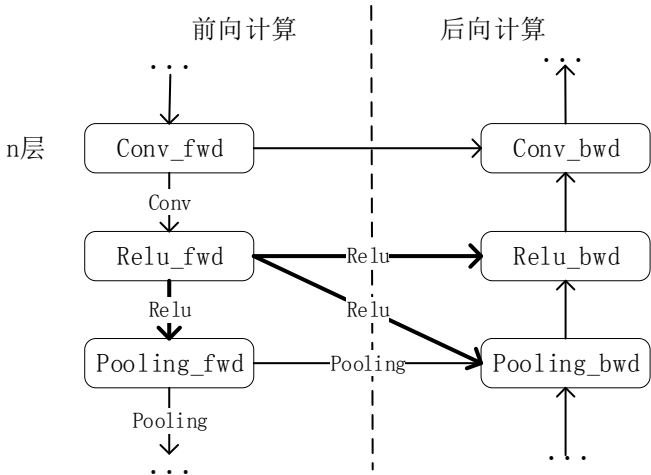


图 4.2 数据流图

在模型训练过程中，节点运算产生的中间结果是需要保留的，因为在计算梯度的时候需要用，这就造成了设备内存的占用问题，而且模型越大，占用的内存就越多，然而设备内存是 GPU 上很稀缺和宝贵的资源。针对此类问题，本文在考虑内存优化的前提下重写计

算图，并提出了基于数据流的内存交换方法，将在后面章节中详细描述。

其次，模型中的依赖控制部分也是实现优化策略的关键，在这种基于图形的数据流优化方法中，数据流交换需要解决两个重要问题：

(1) 特征图换出策略：一个简单的策略是交换所有的功能图，从而实现最佳的内存性能。但是，频繁的数据传输会造成性能的下降。因此，某些特征图仍然可以驻留在 GPU 内存中，以减少开销。需要确定预定义的内存阈值，可以通过对前向子图进行试探性搜索来获得特征映射节点的候选列表。依据链规则导出的节点和损失函数节点之间的关键路径越长，特征图的生命周期越长，因此以反向拓扑顺序进行试探性搜索。基于第 3 章中全局内存的特征参数分析，以 CPU 与 GPU 之间数据传输时间和内存开销作为评价标准，将 GPU 内存预算分配给生命周期较短的计算节点数据流，并将周期较长的数据置换到 CPU 上，这有助于利用计算的时间来掩盖通信时间。通过第 3 章的测试来评估数据传输的时间和节点计算的时间来确定交换阈值。

(2) 特征图换回策略：如果特征图换回太早会导致内存空间被非活动特性图占用，如果太晚则会阻塞下行计算。因此特征映射应该尽可能晚地被换回，并同时保证在计算之前完成数据传输，不要造成数据传输与计算的重叠。实现特征图的换回操作，需要设置触发器来控制该操作的执行，如图 4.1 中来自触发器节点 d 的控制依赖于控制定时换回节点。而触发节点的选择就需要正确的评估计算图中所有节点的完成时间，现有的研究中使用的是加权有向无环图（DAG）算法静态分析的方法完成关键路径的选择，依据单一的节点数量完成时间代价的评估，在本文中对该方法做了改进，通过加入传输和计算性能的评估，使用前向和后向两种搜索算法找到依赖添加节点，以确定张量的换回时间。

在 TFLMS 框架中实现了 Tensor 张量的换出换回功能，该框架提出的目的使深度学习框架尽可能多的处理图像数据。用户可以手动设置需要换出的张量的数目和开始的节点，但是对于具体的张量换出规则没有做详细的规划，换入规则也只是在关联的求导节点的上一个节点添加依赖，将张量换回到 GPU 内存计算，如果当前计算模型中系统的吞吐量不是很高，或者需要传输的数据量很大，而触发依赖的节点中的计算量很小，不足以掩盖数据传输的时间，就会造成求导节点停滞等待数据传输的现象，通过测试发现，在数据传输量很大时，TFLMS 框架处理的数据量提升，然而计算的性能会有下降的趋势。本文通过测试与分析发现：换出张量的数量越多，深度训练模型所能承载的图片数量越多，反而处理图片的性能会很明显的下降。本文从减少片上内存消耗，提高访存性能的角度触发，添加该框架的使用条件，扩展了该框架的功能，与没有优化的算法相比，使用该框架后扩大了 G

PU 的片上内存空间, 缓存的数据增加, 所以数据访问时延减小, 会提高算法的整体性能, 对于想使用该框架做访存提升的用户来说, 有很大的参考价值。本文提出优化的目的是在考虑数据传输代价的前提下, 以换出张量来扩大内存的方式来优化深度学习模型。

实现优化思路的过程为: 通过深度学习算法在 TensorFlow 上的预运行, 得到计算图, 结合第 3 章架构内存的访存特征, 结合 3.3 节的数据传输评估模型, 通过计算图中的数据传输代价建模, 确定基于数据传输代价的数据换出和换回的规则, 以尽可能的保证反向求导计算不会受到数据访问时延的影响。

4.2 数据流计算图建模

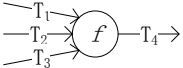
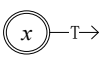
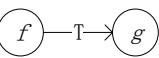
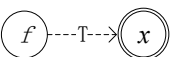
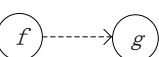
(1) 定义 1. (计算图)

设 $G = (V, E, \lambda, \tau)$ 是顶点和边标记的有向图, 其中 V 是 G 中的顶点集, $E \subseteq V \times V$ 是 G 中边的集合, $\lambda: V \rightarrow (O, \text{Bool})$ 是将每个顶点映射到操作 o 的的函数 (元组 $o \in O$) 和表示操作是否参数化的布尔值, $\tau: E \rightarrow (D, \text{ACT})$ 是将每个边缘映射到数据类型 D 和将动作映射到 ACT 映射函数, 其中 $\text{ACT} = \{ \text{“读取”}, \text{“更新”}, \text{“控制”} \}$ 。本文对数据类型 D 做了进一步封装 $D \rightarrow (T, (t1, t2, t3))$, T 表示在节点之间传递的 tensor 张量, 在 GPU 上的计算节点之间 $t1$ 表示该边连接的前向节点最快执行时间即理论分析时间, $t2$ 表示预运行该节点的执行时间即该节点执行时间最大的消耗。 $t3$ 表示数据传输时间为 0, 在 Cpu 上的边中 $t1=t2=t3=0$, 在 GPU 到 CPU 的连接边上 $t1$ 表示该边连接的前向节点最快执行时间即理论分析时间, $t2$ 表示预运行该节点的执行时间即该节点执行时间最大的消耗, $t3$ 表示数据从 GPU 传输到 CPU 所需要的时间, 在 CPU 到 GPU 的连接边上 $t1$ 表示该边连接的前向节点分配空间所需的时间, $t2$ 表示该边的前向节点数据传输的时间, $t3$ 表示数据从 CPU 传输到 GPU 所需要的时间, 所以将 D 分为三类, 第一类为(GPU,GPU), 第二类为(GPU,CPU), 第三类为(CPU,CPU), 第四类为(CPU,GPU)。

在深度学习中, 计算图用于表达由输入和输出通常是多维数组的神经计算操作组成的网络。用于存储神经网络的内部状态的张量被定期更新, 例如, 神经网络中的隐藏层中的学习权重和偏差。因此, 本文将操作分类为正常操作和参数化操作, 其中参数化操作具有可以更新的状态。变量是一种特殊的参数化操作, 使用标识操作来更新内部变量。常量是变量的特殊情况, 其值设置一次且永远不会更新。每个边都有一个值, 用来表示与该边的张量相关的动作, 有三个动作分别为: “读取”, “更新” 和 “控制”。考虑从操作 u 到操作 v 的边缘 (u, v) , 动作 “读取” 和 “更新” 分别表示 v 读取和更新边上传递的张量, 动作 “控制” 意味着传递的张量信息触发 v 的执行, 称这种操作为控制依赖操作。

表 4.1 列出了计算图不同顶点和边的符号表示。函数组合表示为“ $\circ$ ”，从定义中可以得到 $(f_2 \circ f_1)x = f_2(f_1(x))$ 。函数“ $\cdot_i$ ”是取元组中的第 i 个元素，例如 $(a, b) \cdot_2$ 返回 b 。通常在所有进入边具有数据时触发计算图中的操作执行。该操作完成后在其输出边上生成数据，然后以相同的方式重复触发其他操作。当所有可到达的操作都被执行并且所有可到达的边都被数据填充时，此过程结束。除变量之外的每个可达操作被执行一次。在计算开始时，如果没有为边设置值，则无法触发图形的执行，计算图是非循环图，并且有一些操作没有传入边，无法触发这些操作，可通过使用变量解决此问题。计算图中的变量用于存储可学习的参数，输入和输出数据，并用于触发图的计算。变量的特殊应用是将深度学习计算图与一般图区分的重要特征。

表 4.1 计算图语义理解

	图	定义	说明
节点		$\lambda(f) = (f, False)$ $T_4 = f(T_1, T_2, T_3)$	节点操所为 f ，输入为 T_1, T_2, T_3 ，通过运算输出 T_4
		$\lambda(x) = (x, true)$ $T = x$	变量 x ，参数化操作
边		$\tau(f, g) = (T, "read")$	f 输出 T ， g 读取 T ，边表示 $g \circ f$
		$\tau(f, x) = (T, "update")$ $x = T$	f 输出 T ，并用 T 来更新 x ， T 和 x 必须是相同的类型
		$\tau(f, g) = (_, "control")$	f 触发 g 的执行

(2) 定义 2. (拓扑排序)

给定计算图 $G = (V, E, \lambda, \tau)$ ，令 N 为图中顶点的数量，拓扑排序是顶点到整数的映射， $\gamma: V \rightarrow \{0, 1, \dots, N-1\}$ ，满足

- $\gamma(v) = 0, \forall v \in V \wedge \lambda(v) \cdot_2 = true$
- $\gamma(u) < \gamma(v), \forall (u, v) \in E \wedge \lambda(v) \cdot_2 = false$ 。

通常，拓扑排序表示图中操作的执行顺序。给定两个操作 u 和 v ，如果 $\gamma(u) < \gamma(v)$ ，则在 v 之前执行 u 。如果 $\gamma(u) = \gamma(v)$ ，则 u 和 v 并行执行。在本文中，变量总是为 0，这意味着将首先执行变量，并且它们的传入边（“更新”边）不会改变它们的顺序。稍后执行变量取决于其传入操作，并且与变量的顺序无关，仅这些执行不会触发变量的传出操作。

图 4.3(a)具有以下执行顺序：“ $x \rightarrow h \rightarrow f \rightarrow x$ ”。首先，变量 x 由用户初始化，然后触发操作 h ，接着，执行 h 并触发操作 f 。最后，用 f 的输出更新 x ，计算结束。操作 h 仅取决于 x ，而 x 本身不能再次触发 h 。

图 4.3(b)中的示例图可以具有两种可能的执行顺序：“ $x \rightarrow h \rightarrow f \rightarrow x \rightarrow f_1$ ”，或“ $x \rightarrow h \rightarrow f \rightarrow f_1 \rightarrow x$ ”。基于张量 T_1 和 T'_2 都可用时触发操作 f_1 。很容易看出 f_1 必须在 f 和 x 之后执行。但是， x 被执行多次。重要的是要知道 x 的哪个输出用作 f_1 的输入。为避免歧义，本文提出以下关于变量的约定：第一，操作始终使用变量的最新值；第二，在使用相同张量的操作中，变量始终具有最高的执行优先级。

此约定有助于本文确保在使用 f 的输出更新 x 后再执行 f_1 。操作的执行顺序不仅取决于传入边的数据可用性，还取决于控制依赖性边。“控制”边没有数据。换句话说，它们不是操作的输入。“控制”边用于控制操作的执行顺序。在图中添加“控制”边将改变图的拓扑排序。如果 (u,v) 是“控制”边，则必须在 u 之后执行 v ，并且 $\gamma(u) < \gamma(v)$ 。根据这个定义，变量没有控制边。

图 4.3(c)中的示例图增加一个新操作 f_2 ，其使用 f 的输出，执行计算并更新变量 x 。在没有从 f_2 到 f_1 的控制边的情况下，在执行 f 之后， f_1 和 f_2 可以并行执行，因为它们不相互依赖。因为它们都访问变量 x ，即 f_1 读取 x 并且 f_2 写入 x ，所以需要控制边缘以确保它们按顺序访问 x 。从 f_2 到 f_1 的“控制”边表示 f_1 将在完成 f_2 并更新 x 后执行。

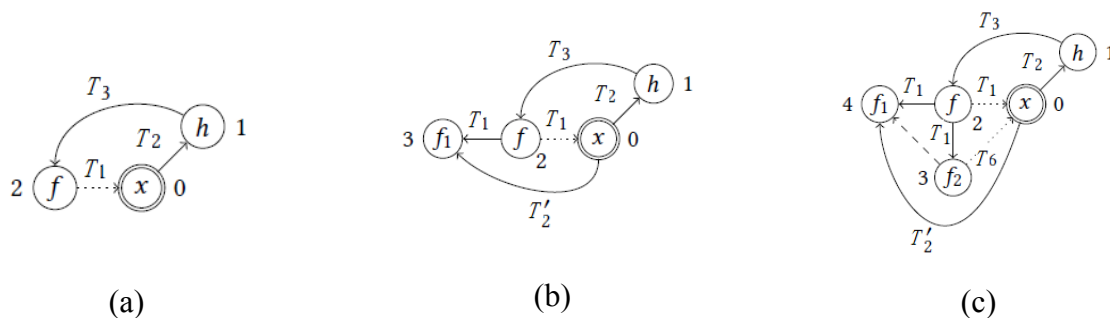


图 4.3 计算图拓扑

(3) 更新可学习的参数

训练神经网络涉及最小化目标函数 f ，以衡量预测结果与实际值之间的差距。目标函数是具有可学习参数的多个函数的组合，梯度下降算法经常用于最小化函数。优化是一个更新可学习参数的迭代过程，以最小化目标函数，其中每个训练迭代包括三个阶段：前向阶段计算目标函数，后向梯度计算阶段，以及使用梯度更新可学习参数的阶段。在迭代开始时，清除可学习参数的以外的张量，输入的张量被更新并触发迭代。

图 4.4 显示了在训练期间更新可学习的参数（由变量表示）的过程。在前进阶段，变量

x_i 是函数 f_i 的输入， f_i 的输出在后面的函数中使用，最后由目标函数 f 产生损失值。在后向阶段，本文根据可学习的参数计算 f 的梯度，需要 f_i 的输出作为函数 $\nabla_{x_i} f$ 的输入来计算 f 相对于 x_i 的梯度，最后，在更新阶段使用函数 U_{x_i} 更新 x_i 。

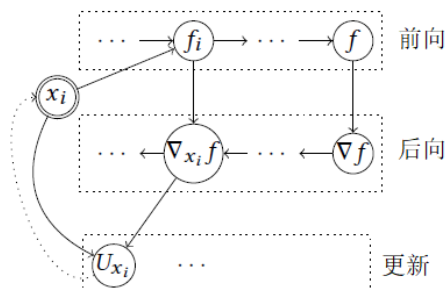


图 4.4 后向计算过程

（4）张量的生命周期：

如果 TensorFlow 中不再使用张量，则通过 TensorFlow 垃圾回收而释放。每个张量都被分配一个引用计数，即操作数。每次操作使用完张量后，其引用计数减 1。如果引用计数达到零，则释放张量的内存空间。换句话说，张量的寿命是从产生它的操作到使用它的最后一个操作。令 T_s 为由操作 u 产生的张量，并且 $v_1, v_2, \dots, v_k$ 是使用 T_s 的 k 个操作。 T_s 的寿命计算为 $\max\{\gamma(v_1), \gamma(v_2), \dots, \gamma(v_k)\} - \gamma(u)$ 。

在本文中，通过前一章的探测分析，对原有的计算图模型做了改进，在本文中，张量的寿命定义为： $\max\{Time_{all}(v_1), Time_{all}(v_2), \dots, Time_{all}(v_k)\} - Time_{all}(u)$ ，这样的定义与之前相比更加准确， $Time_{all}(v)$ 表示程序执行到节点 v 花费的时间总和。

4.3 重写计算图

受到了 GPU 有限的存储空间的限制，一个很大的神经网络模型在训练时经常会发生内存不足错误。这主要是因为计算图中存在许多具有长周期的张量。在本节中，将展示如何重写图，以更好地使用有限的 GPU 内存对其进行训练。本文的想法是暂时将 GPU 中的“长寿命”张量发送到 CPU，并在必要时将它们发送回 GPU。

图 4.5 展示了计算图重写示例。图 4.5(a) 中的粗边被重写以生成图 4.5(b)。顶点上方或下方的整数是拓扑排序中该顶点的顺序，顶点 f_1 被换出，因此重写从 f_1 到 f_2 和 f_3 的边，图 4.5(c) 中 f_i 和 f_j 是控制依赖性操作，分别触发交换操作 $id3$ 和 $id4$ 的执行。为了将张量存储在 CPU 内存中，本文派生操作以自动将张量发送到 CPU 并将其发送回 GPU。

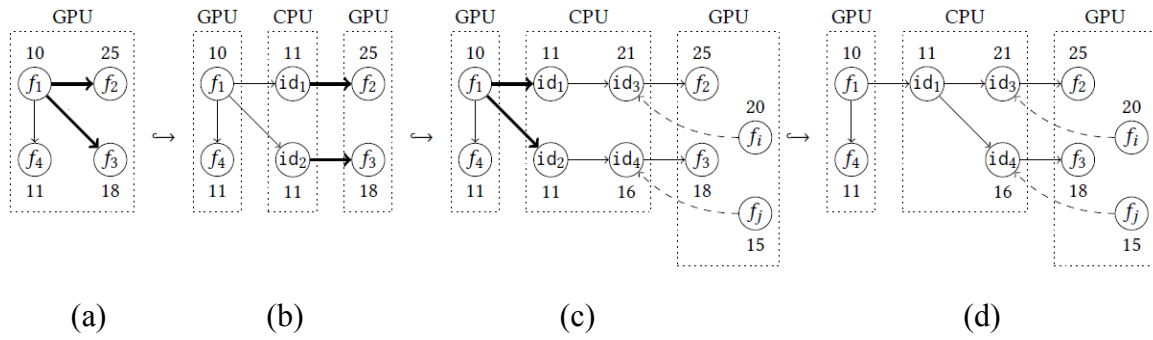


图 4.5 计算图重写实例

如果考虑边 (f_1, f_2) ，其中使用 GPU 执行 $\tau(f_1, f_2) = (D, _)$ ， f_1 和 f_2 。该边的计算可表示为

$$f_2^G \circ f_1^G \quad (4-1)$$

上标 G 代表 GPU 上的操作，将此计算重写为：

$$f_2^G \circ id^C \circ f_1^G \quad (4-2)$$

等式 4-2 中的函数 id 是换出操作，其中上标 C 代表 CPU，而 id 是一个自函数，即 $id(x) = x$ ，换入和换出的向量值不变。由于 id 是使用 CPU 执行的，因此 f_1 的输出张量在 f_1 结束后立即换出到 CPU 内存中，并释放 GPU 内存。当 f_2 被触发时， id 的输出张量将被换入 GPU，输入到 f_2 中。

使用公式 4-2，本文可以重写图形，从而减少 GPU 内存消耗。但是，考虑到系统的带宽问题，如果连续不断的将张量换出的话，会增加当前平台的负载，所以，本文提出了间隔换出张量的策略，而且，对于其中的边 (v, u) ， v 在 u 之后立即执行。因此，不需要在这样的边上交换张量。接下来本文将介绍基于数据流代价的计算图重写条件，并定义阈值 α ，如果同时满足以下条件，则 u 的张量可换出到 CPU。

$$\begin{cases} \alpha \leq \gamma(v) - \gamma(u) \\ \alpha \geq n \leftarrow trans(u) \\ \alpha' \leq \gamma(u) - \gamma(u_fwd) \end{cases}, \quad (4-3)$$

其中 $n \leftarrow trans(u)$ ，表示 u 换出换入总的时间代价，并依据上一章预运算的计算时间评估值，找出该时间间隔之内的节点数 n ， α' 的含义是当前的节点 u 与上一个重写的节点之间的间隔节点数， α' 的值需要依据当前深度学习模型的执行情况做适当的调整，通过测试获得最优值。

4.4 数据流内存优化模型改进

若直接应用公式 4-2 存在以下两个问题：如果换回张量的时间过晚， f_2 必须等待从 CPU 内存发送到 GPU 的张量；可能出现多次换回张量的问题，因为除了 f_2 读取它之外可能存在多个其他操作。所以在本文中，使用以下三种改进方法：首先重新定义 CPU 上的操作函数；然后将相同张量的换出操作融合到单个换出操作中；最后提出基于计算和传输代价的张量换回策略。

(1) 函数重定义

如图 4.5(c)所示，换入和换出操作不能是同一个函数表示，要尽可能早的换回张量，需要添加一些额外的操作。将 id 函数重写为两个函数的组合，即原函数和反函数，选取 f 作为 id 的函数：

$$id = f^{-1} \circ f \quad (4-4)$$

公式 4-2 可表示为：

$$f_2^G \circ f^{-1C} \circ f^C \circ f_1^G \quad (4-5)$$

如果想减少 CPU 上的内存消耗，可以使用一对编码和解码函数来代替 id 。

$$f_2^G \circ id_2^C \circ id_1^C \circ f_1^G \quad (4-6)$$

在等式 4-6 中， id_2 将用于将张量换回到设备，将函数 id_2 称为换回操作。而且，必须以较优的顺序手动触发 id_2 ；否则， id_2 就会在 id_1 之后立即执行。为此，必须添加从某一操作到 id_2 的控制边，本文提出了考虑计算和传输代价的优化换回策略。

(2) 融合操作策略

考虑多个交换操作多次交换张量会消费带宽的情况。如果张量很大并且操作彼此接近，最好将交换操作融合到一个交换操作中。张量仅换回一次并驻留在 GPU 内存中以供消耗操作重用。例如，在图 4.5(d)，如果 f_2 和 f_3 接近且传输的张量很大，那么本文将 id_3 和 id_4 融合成一个单一的操作。为了确定两个操作的接近程度，本文可以为它们之间的距离定义一个阈值 β 。 β 的确定： β 表示两个操作的接近程度，本文中参考文献[56]将其设定为 3。

(3) 基于运算传输代价的换入策略

本文通过 3.3 节的数据传输建模，确定了数据传输的时间代价，通过 3.4 节的深度学习性能分析，确定理论上节点计算的时间，通过对算法模型的预运行确定实际计算时间实现计算图重写，实现内存的换出换回策略的关键是添加节点之间的控制边，尽可能准确的确定边上数据传输的权值，设计合理的控制边添加算法。

将交换操作的控制边添加到计算图节点中用于触发交换操作，以减少交换通信的开销同时保证计算图的等价性，考虑公式 4-6，交换操作 $id2$ 的控制操作必须从一组操作 V_c 中选择，其中对于 $\forall v \in V_c$ ，都存在 $\gamma(id1) < \gamma(v) < \gamma(f_2)$ 以保证计算图正确性。设 $k = \gamma(f_2) - \gamma(v)$ 为 f_2 和 v 的距离。如果 k 太小，张量的交换太迟了， f_2 可能等待张量。如果 k 太大，则交换张量太早了，张量将在设备中保存了很长时间直到被 f_2 使用。

选择控制操作的理想解决方案是建立计算图的代价模型并使用该模型优先操作节点。但是，在 TensorFlow 中，一个操作的输入输出张量形状通常是未知的，除非将数据输入计算图然后触发操作。这意味着，在重写图形时，没有关于张量的实际大小的信息，只能静态计算操作成本。所以原有的张量换出换回模型有很大的缺陷，虽然能够很有效地增加深度学习模型训练的图片数量，但是以牺牲运算性能为代价的。本文的优化方案建立在静态理论分析和预运行的实际数据基础上，引入两个参数：下限 σ_l 和上限 σ_u ，并定义了节点添加依赖边的条件。

首先，确定搜索节点的上下限。 f_i 节点的张量换出并换回的代价为 $trans(f_i)$ ，假设后向操作为 f_i ，且 f_i 满足换出条件，从后向节点开始向前搜索，按照预运行前情况下节点的实际运行时间，确定可以掩盖数据换回到 GPU 的计算节点的数量，即为下限 σ_l ：

$$\begin{cases} \sigma_l = \gamma(f_0) + n_{exe} \\ trans(id1) \leq time_{exe}(f_0 + f_{0+1} + \dots f_{0+n_{exe}}) \end{cases} \quad (4-7)$$

其中 $time_{exe}$ 表示节点的预运行时间， n_{exe} 表示在预运算状态下可选的节点总数。上限 σ_u 可通过以下过程求的：

$$\begin{cases} \sigma_u = \gamma(f_i) - n_{theory} \\ trans(id1) \leq time_{theory}(f_i + f_{i-1} + \dots f_{i-n_{theory}}) \end{cases} \quad (4-8)$$

其中 $time_{theory}$ 表示节点的评估运行时间， n_{theory} 表示在评估模型理论分析的情况下可选的节点总数。因为代价的建模是理想化的情况，每个节点的实际运行时间会随着系统的整体性能的变化，出现波动，并且本文优化的目的是在换入还出内存之后，保持深度学习模型的性能不会下降，最好的优化结果是在基于代价模型的基础上，与不使用框架的算法相比性能有所提升，这个时候，每个计算节点的运行时间就会减少。如果严格的以预运行的时间来计算会影响实验的结果，所以本文的上下界可以通过具体的实验情况做出调整。

其次，定义依赖节点上的最大换回数据量为系统的最大吞吐量。使用函数 $Thr(f)$ 表示节点 f 上添加的依赖数据量， $Througp_max$ 表示最大吞吐量，如果为当前节点添加数据换回依赖，那么当前节点需要满足：

$$Thr(f) < Througp_max \quad (4-9)$$

为了保障当前系统的负载，使用系统最大吞吐量作为依据，添加数据换回依赖，同时从 CPU 到 GPU 的数据传输量，不能超过系统最大的吞吐量，以保证性能不受到数据换回的影响。

(4) 算法的实现

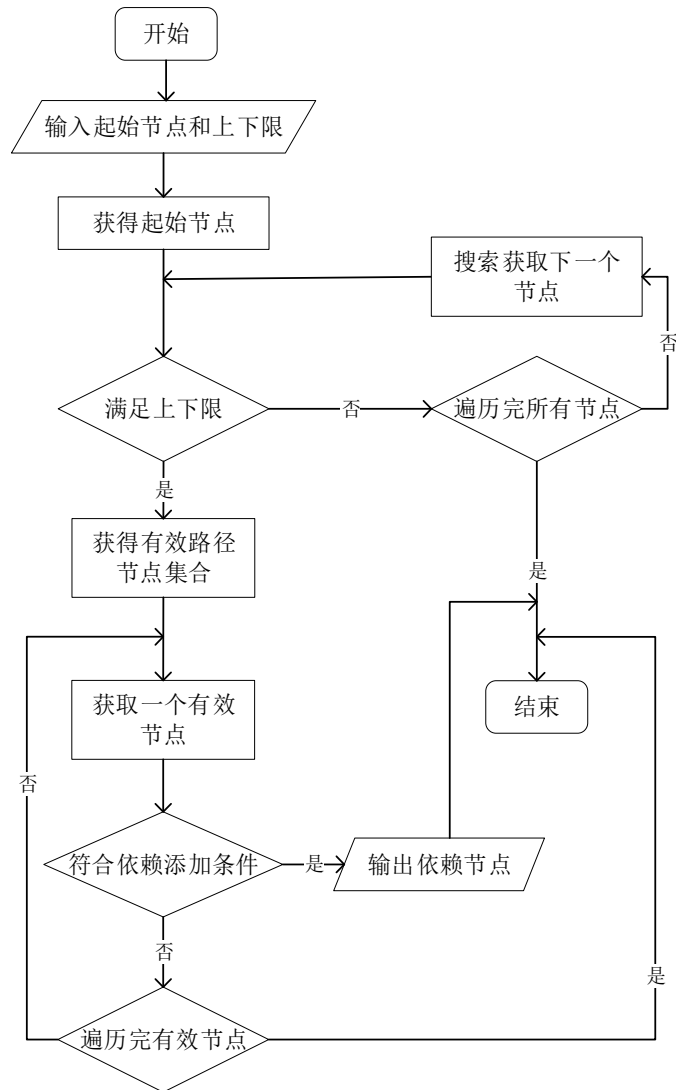


图 4.6 依赖节点查找流程图

在通用的情况下，假设一个边 (f_1, f_2) 使用换出重写操作，换回操作为 s_i ：

$$f_2^G \circ s_i^C \circ s_0^C \circ f_1^G \quad (4-10)$$

关于 s_i 的查找流程如图 4.6 所示，首先找到满足上下限范围内的节点，然后确定一条可达的路径，其次依据代价函数判断节点是否满足添加以来的条件，依据查找节点的方向不

同，即接近上限或者接近下线程度的不同，分为后向和前向两种实现策略。

本文提出的两种控制查找添加策略。一种是后向拓扑查询策略。在目标操作 f_2 和重写的操作 f_1 之间，直接使用拓扑排序来获得可控制操作的一组候选节点，下限和上限是相对于 f_2 。算法 1 显示了该策略的算法。候选节点是与 f_2 的距离在 σ_l 到 σ_u （第 7 行）范围内的操作，存在从它们到 f_2 的路径（第 8 行），并且该节点当前所添加的数据依赖数据量没有超过系统每秒最大吞吐量，一旦找到满足上述条件的一个操作（第 10-13 行），算法就会停止，通过该算法获得的控制节点是最晚触发数据换回的情况。具体的实现过程描述如下：

算法 1：后序拓扑查询

输入：需要换出的节点 f_1 ，关联的后向求导节点 f_2 ，搜索下限 σ_l ，搜索上限 σ_u

输出：一个触发 f_1 输出参数对应的 CPU 上节点换回的操作节点 g

```

1:  $l \leftarrow \max\{\gamma(f_2) - \sigma_u + 1, \gamma(f_1)\}$ 
2: for  $i \leftarrow \sigma_l$  to  $\sigma_u$  do
3:    $k \leftarrow \gamma(f_2) - i$ 
4:   if  $k \leq l$  then
5:     return null
6:   end if
7:    $queue \leftarrow \{f \mid f \in V, \gamma(f) = k\}$ 
8:    $queue \leftarrow \{f \mid f \in queue, f_2 \in Reach(f)\}$ 
9:    $queue \leftarrow \{f \mid Thr(f) < Througp\_max \cdot ls\}$ 
10:  if queue is not empty then
11:     $g \leftarrow GET(queue)$ 
12:    return  $g$ 
13:  end if
14: end for

```

第二种是前向链式搜索策略。链规则策略从源操作 f_1 开始沿着计算方向查找，以找到对应的后向操作作为控制操作的候选者。广度优先搜索用于遍历前向阶段中的操作，其中下限和上限是相对于 f_1 。算法 2 显示了该策略的具体过程。对于广度优先搜索策略来说，本文设置了两个开集 s_1 和 s_2 ，以及一个闭集 s_c ， s_1 包含当前的正向操作， s_2 包含下一级的正向操作（包括 s_1 中的所有操作的传出操作）， s_c 包含访问过的操作，从 f_1 开始，如果算法在 σ_l σ_u （第 8 行）的范围内获得当前操作的输出后向操作（第 9 行），就检查这些后向操作的有效性（第 10-11 行）。如果存在一个有效操作，则当前节点就是候选节点并且算法返回它。否则，算法进入下一循环（第 27-30 行）。

算法 2：前向可行遍历

```

1:  $S_1 \leftarrow \{f_1\}; S_2 \leftarrow \Phi; S_c \leftarrow \Phi$ 
2: while  $S_1$  is not empty do
3:   if  $\sigma_u = 0$  or  $\sigma_l > \sigma_u$  then
4:     return null
5:   end if
6:    $s \leftarrow GET(S_1)$ 
7:    $op \leftarrow Out(s)$ 
8:   if  $\sigma_l \leq 0$  then
9:      $B \leftarrow \{f \mid f \in op, f \text{ is a backward operation}\}$ 
10:     $B \leftarrow \{f \mid f \in B, \gamma(f_1) < \gamma(f) < \gamma(f_2)\}$ 
11:     $B \leftarrow \{f \mid f \in B, f_2 \in out(f)\}$ 
12:    if  $B$  is not empty
13:       $g \leftarrow GET(B)$ 
14:      return g
15:    end if
16:  end if
17:   $N \leftarrow \{f \mid f \in op, f \text{ is a forward operation}\}$ 
18:  for  $f$  in  $N$  do
19:    if  $f \in S_c$  then
20:      continue
21:    end if
22:    if  $f \notin S_2$  then
23:       $S_2 \leftarrow S_2 \cup \{f\}$ 
24:    end if
25:  end for
26:   $S_c \leftarrow S_c \cup \{s\}$ 
27:  if  $S_1$  is empty then
28:     $\sigma_l \leftarrow \sigma_l - 1; \sigma_u \leftarrow \sigma_u - 1$ 
29:     $S_1 \leftarrow S_2; S_c \leftarrow \Phi$ 
30:  end if
31: end while

```

4.5 本章小结

本章在 TFLMS 模型的基础上,结合第 3 章的性能评价方法提出了基于代价的数据流交换方法,并通过内存融合策略和前向后向两种搜索方式进一步改进了该方法。本文在原 TFLMS 模型中引入了数据传输和运算代价,在两者的限制下确定张量换出到 CPU 的条件,并且通过代价模型找到张量从 CPU 换回到 GPU 的时间范围,通过前向和后向两种搜索方式实现了依赖节点的查找。本章的数据流交换方法可以从优化访存的角度,提升模型计算的性能,与原 TFLMS 模型相比,在保证数据交换的及时性和计算的高效性方面更有优势。

第5章 实验验证与结果分析

本章中将对第 3 章和第 4 章的主要实验思路与实验结果做分析，验证优化策略的有效性。实验环境参数如表 5.1 所示：

表 5.1 实验环境参数

平台	核心频率	内核数量	CPU-GPU 总线类型
GTX 1080 Ti	1.72 GHz	3584	PCIE3.0
	TensorFlow 版本	Cuda 版本	Python
	1.10.1	9.0	2.7

5.1 实验方案

本文研究的最终目标是实现基于代价的数据流交换模型，该模型的前提是实现深度学习的性能评估，主要包括数据传输模型的评价和预运行性能剖析。最终，在 TFLMS 模型的基础上加入优化策略进行实验对比分析。

（1）数据传输评价模型实验

数据传输评价模型的目的是评估内存换出换回所产生的额外的代价，并通过合理的交换策略的设计来避免或者减少代价的产生，所以本文通过大量的基准测试统计不同数据量的 GPU/CPU 传输时间，然后，通过实际测试与模型评估的方法验证了本文所使用的 Log Gp 数据传输模型的合理性。

实验方法：首先，使用一个 `cudaStream` 在 10^6 到 1000Mb 的数据量之间，使用 `cudaMemcpy` 和 `malloc` 方法在记录 CPU 和 GPU 之间传输数据的平均时间，然后，使用不同的数据传输量，控制多个 `cudaStream`，记录实际的数据传输时间，并与理论分析时间做对比。

（2）深度学习算法预运行性能剖析实验

深度学习模型性能评价实验主要目的是通过使用 TensorFlow 上的性能测试工具，预深度学习模型运行，得到模型的计算图，并获得每个节点的实际执行时间，并观察实际运行时间，与理论峰值之间的差距。

数据集的选取：`mninet` 手写数字数据集。`mninet` 数据集是诸多深度学习模型优化中使用的标准数据集，便于实现实验结果的对比验证。

深度学习模型的选择：本文的实验选用 ResNet-50 模型。ResNet 网络是参考了 VGG19 网络，在其基础上进行了修改，并通过短路机制加入了残差单元，主要变化主要体现在 ResNet 直接使用 stride=2 的卷积做下采样，并且用 global average pool 层替换了全连接层。ResNet 的一个重要设计原则是：当 feature map 大小降低一半时，feature map 的数量增加一倍，这保持了网络层的复杂度。相比普通网络每两层间增加了短路机制，这就形成了残差学习，并且还可以构建更深的网络，当网络更深时，其进行的是三层间的残差学习。这个特性有助于与本文分析内存特征，并且可以保证模型本身计算的准确性。

（3）数据流交换方法实验

为了验证本文的数据流交换优化方法，实验主要包括以下三个方面的内容：

首先，在原模型的基础上做对比实验，测试在不同的批处理数量下模型的性能，默认情况下，使用融合换回策略，依赖控制边的查找策略使用前向可行查找策略。本文测试了原模型的实验结论，并且在原始模型的基础上使用本文的优化方法，与原模型做对比分析，从而论证本文优化方法的有效性。

其次，在原模型的基础上，测试本文提出的融合操作策略和两种搜索策略对实验性能的影响，换出的张量数量固定为全部，因为换出张量的数量越多，对优化策略性能的体现更加明显，变化不同的批处理数量，记录实验结果。

最后，为了验证本文优化模型的通用性，本文换用 VGG-19 网络和 mnist 数据集，针对原 TFLMS 模型和优化模型在换出不同的张量情况下做实验对比。

5.2 数据分析

5.2.1 数据传输评价模型数据分析

图 5.1 显示了使用 cudaMemcpy（固定内存）和 malloc（非固定内存）在各种传输不同的数据量情况下的数据传输时间。对于小的数据量，有大约 10 微秒的恒定开销，在大约 8 Kb 时，传输时间开始线性增加，固定内存的性能优势变得更加显著。对于固定内存，传输方向（进出 GPU 内存）对传输时间没有明显影响，对于非固定内存，在传输量大于 10Mb 时，传输方向对数据传输时间没有明显影响。图 5.2 显示了在 GPU 上函数分配内存所花费的时间，对于小量的数据内存分配，每次调用 malloc 函数的时间小于 0.1 微秒。这个时间在 512B 与 4Kb 之间比较稳定，保持在大约 1 到 2 微秒之间。

在整体上对数据传输代价有一定把握之后，本文通过改变数据流的方式，对第三章中的数据传输模型做了进一步验证，如图 5.2 所示，图 5.2 分多个 Cuda 流传输 16Mb 数据的时间性能。

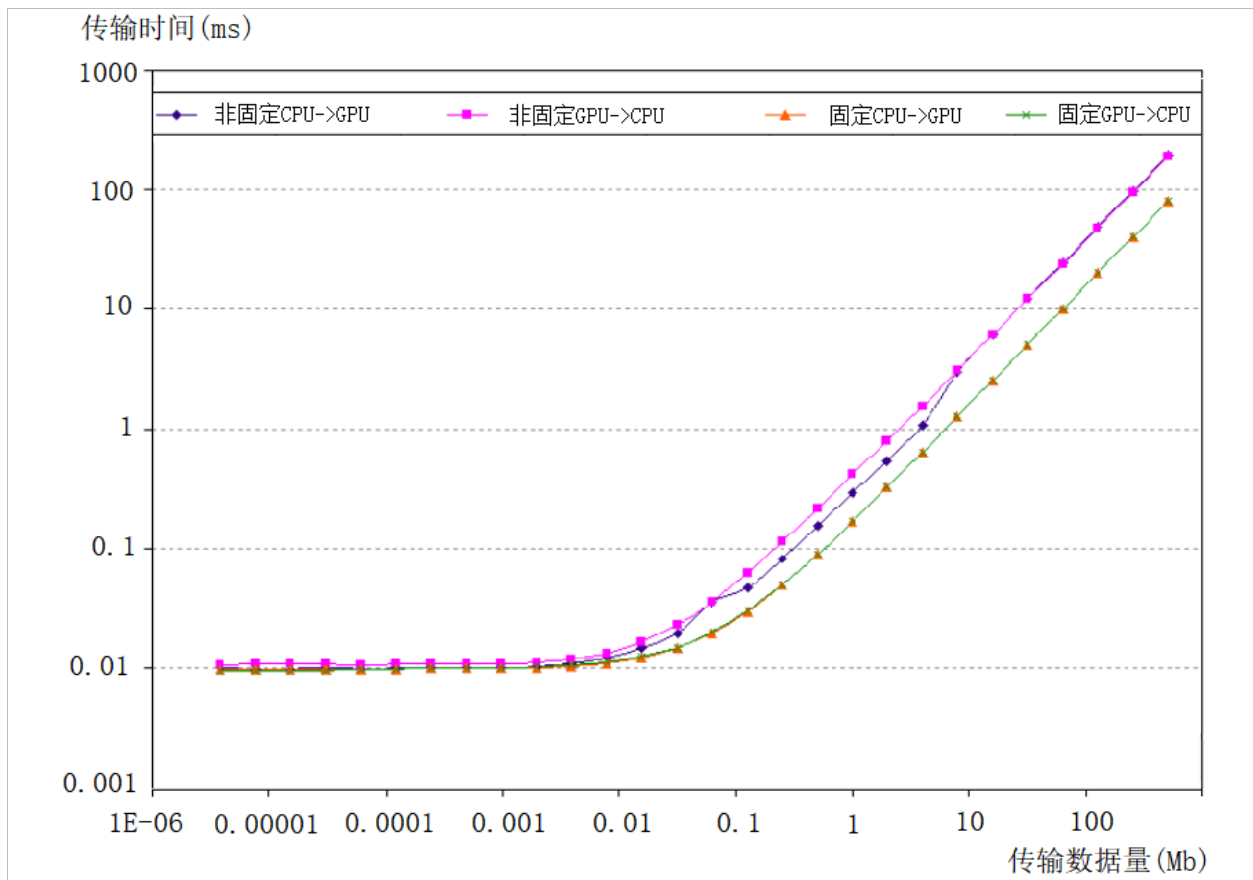


图 5.1 数据传输实验结果

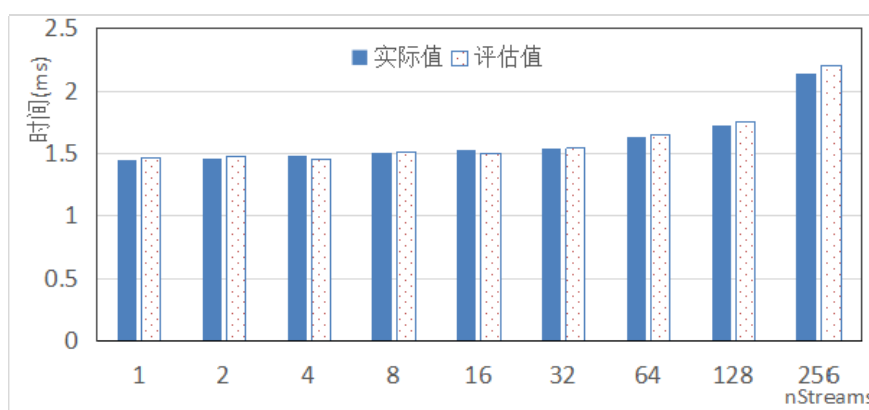
图 5.2(a)是 16Mb 数据从 GPU 传输到 CPU 所用的时间统计，图 5.2(b)是 16Mb 数据从 CPU 传输到 GPU 的时间统计，通过对相同数据量的多个流传输实验发现以下结论：

首先，本文的数据传输模型与实际传输数据相比，差距很小，进一步验证了本文数据传输模型的准确性；其次，理论计算结果与 Cuda 流实际实验数据的对比分析发现，在 32 个流之内，多个流的传输时间变化不大，几乎可以忽略，可以使用一个平均值表示；最后，通过上图的实验数据和查阅相关资料发现并不是多个流都能够并发执行，当超过 32 个之后，传输时间明显增加。在本文的研究中默认的情况是，多个流传输时间相同。

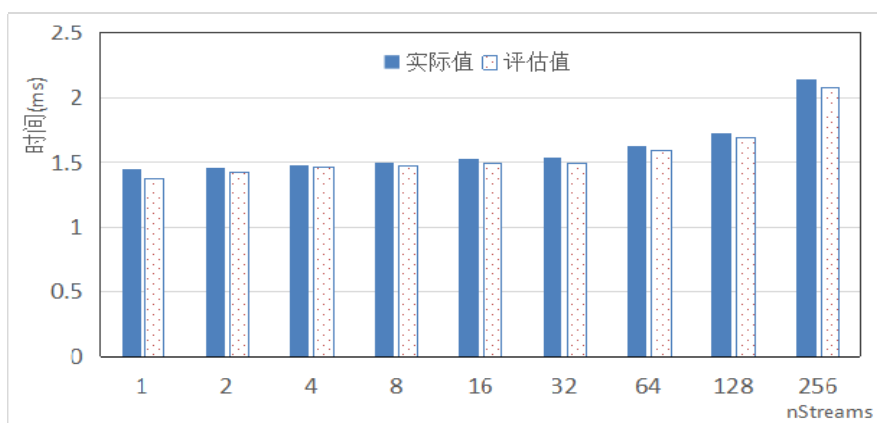
对本文提出的数据访存和性能评估方法的实验数据分析发现，本文改进的 LogGP 数据传输模型与实际测试数据传输时间相符合，可以很好的对实验平台上的数据传输建模。Cuda 语言多个流的并发传输与单个流的传输时间相当。

5.2.2 深度学习算法性能评价数据分析

以 $100 \times 1 \times 28 \times 28$ 作为输入，通过多次实验总结分析发现，前向计算时间约为 56ms，后向计算时间约为 105ms，总运行时间为 161ms。计算图的操作节点数量为 10000。平均一个节点的运行时间为几个数量级 μs 。但是前向计算过程中节点的计算时间是递减的，而后向节点的计算时间有递增的趋势。并且后向节点的计算时间比前向节点长，张量数量为 1000 个。在不同的 batch 下张量的大小不同，同一个 batch 情况下，后向计算的张量偏大，这与 ResNet 网络自身的参数结构体现的特点一致。在网络计算的各个阶段，内存使用情况和节点的计算时间实验结果如下图 5.3，5.4 所示。



(a) GPU 到 CPU 传输时间



(b) CPU 到 GPU 传输时间

图 5.2 多个流数据传输实验结果

在图 5.3 中可以发现，除了 res3a 计算过程中后向计算占用内存比前向计算的内存占用大两倍之外，在其他过程中前向计算和后向计算阶段内存占用情况相差不大，内存使用情况呈现先减后增的趋势。图 5.4 中发现，在计算节点，前向计算时间和后向计算时间在总体上都是递增的趋势，并且后向计算时间整体上高于前向计算时间。使用 Tensorboard 工具

对节点占用内存和运行时间的分析目的是验证剖析深度神经网络内部运行情况的方法，通过实验发现，Tensorboard 工具可以获得详细的节点输入输出张量大小和节点的实际运行时间等量化指标，为本文的下一步工作做了准备，实验中发现节点全部展开后，每个节点的执行时间是 μs 数量级的，通过间隔多个节点的依赖添加可以覆盖张量换入换出的时间，如果以网络层为粒度展开计算图，每一层输出到后向计算的张量数量为 300 到 700 个，张量的换出可以在不同的网络层之间触发，张量的换回可以从后向计算节点开始查询，以最小的粒度添加节点依赖。

本文通过使用 Roofline 模型对深度学习模型做理论运算峰值建模，并与实际运行情况对比发现，实际的运行性能最高只能达到计算峰值的 70%。以每张图片的处理时间为对比指标，记录在不同的批处理值情况下模型的运算性能如表 5.2 中所示。

表 5.2 模型性能对比

	1	16	32	64	128	176
预运行时间(s)	60	101	174	217	247	248
理论分析时间(s)	45	74	118.32	152	172	170

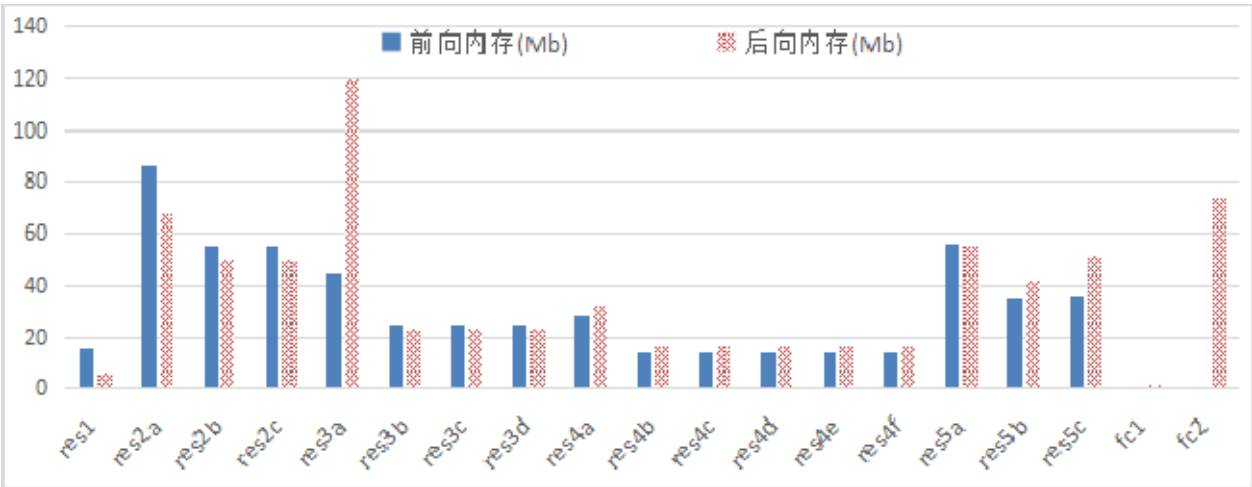


图 5.3 深度学习模型消耗内存实验结果

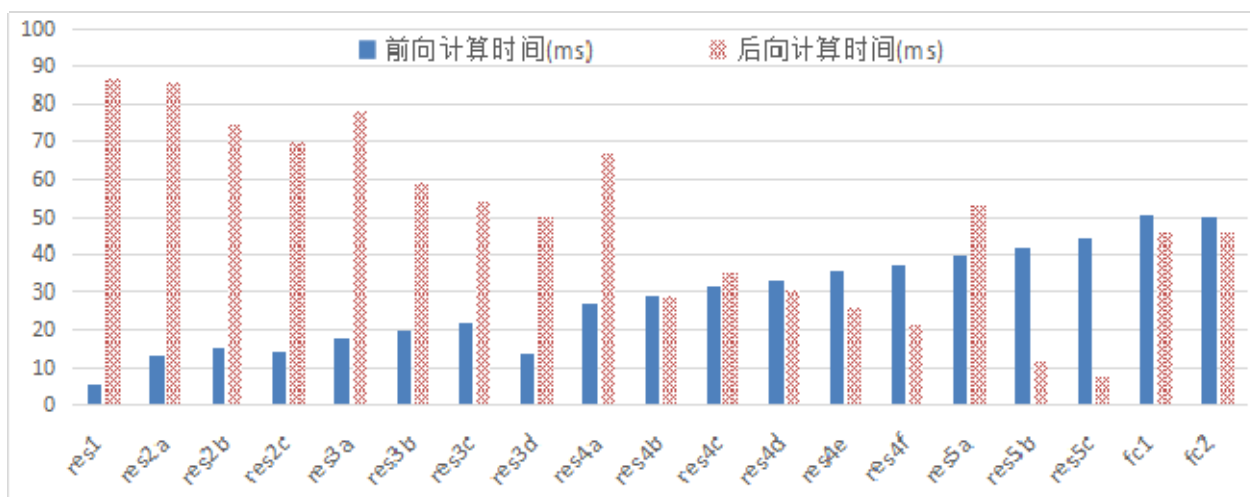


图 5.4 深度学习模型计算时间实验结果

Tensorboard 工具对深度学习模型的数据流计算图、节点的张量信息、内存、耗时情况的统计分析，为确定数据流换出换回时机提供依据。Roofline 理论计算模型与预运行性能对比发现，深度学习模型实际性能远低于理论峰值，性能瓶颈体现在数据访存上。

5.2.3 数据流内存交换方法数据分析

(1) TFLMS 原模型实验验证

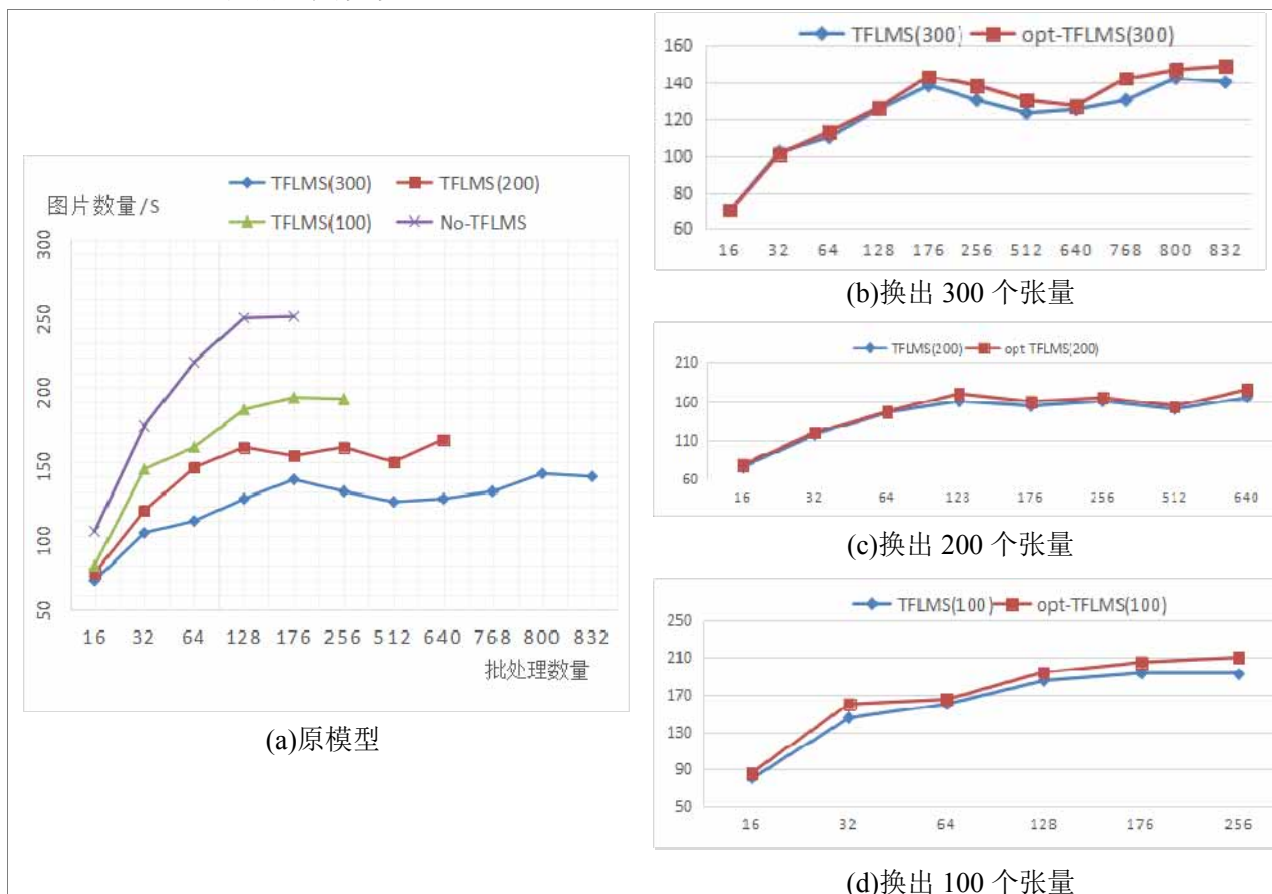


图 5.5 与原模型实验数据对比

图 5.5(a)为文献[17]中原 TFLMS 模型的实验结果，从图中可以发现换出张量后模型的性能明显下降，本文使用基于代价的数据流交换方法与文献[17]做了对比实验，如图 5.5(b), (c), (d)所示。从实验数据中总结以下结论：与没有换出张量的计算过程相比所有使用换入换出框架的实验中，整体性能都是下降的趋势，这与该框架的设计思路相符合，主要目的在于扩展训练的数据量；在换出张量数量相同的情况下，分别对 100，200，和 300 个换出张量的情况，与原模型做了对比，本文的优化模型比原模型的体现更好的性能优势；本文从换出内存增加访存性能的角度考虑，所设计的基于代价的访存优化方案，比原始模型的性能有所提升，但是在批处理量很小的情况下并不能体现优势，当批处理数量增加到 32 时性能开始有明显的提升，本文设计的优化模型，曲线有上升的趋势；系统的最大吞吐量是一定的，当换出的张量很多时，反而会影响运算的速度，使算法的整体性能下降。

（2）改进策略验证

如图 5.6 所示，本文还对融合内存的处理方式做了多次实验验证，通过本文的实验发现使用该优化策略只在最开始的一个范围内有优势，随着数量的不断增加，不使用融合存储的方案性能略有提升，这与 CPU 与 GPU 之间以多个流的方式传输数据有关，当合并内存的存储量很大时，内存中的数据需要串行发射，会影响数据传输性能，然而，当数据在 CPU 上没有合并存储时，数据更容易实现并行发射充分利用数据传输带宽，所以随着数据量的增加，最好的方式是不使用数据在 CPU 上的合并存储方案。

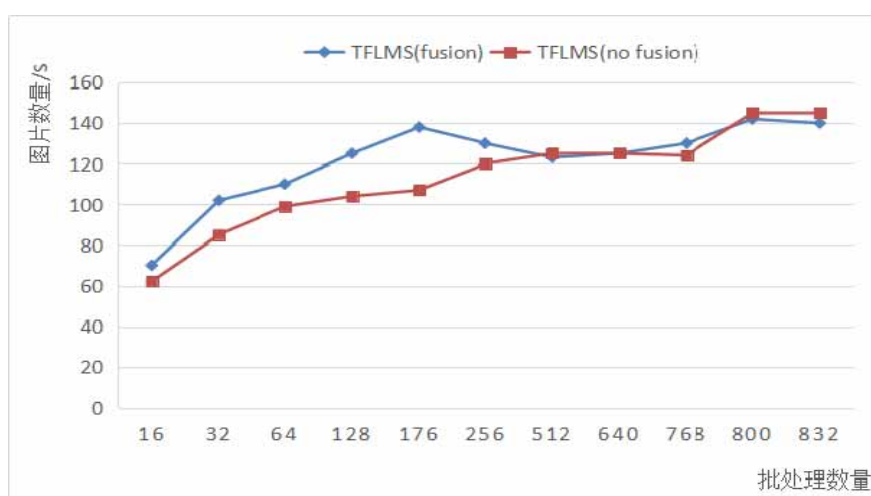


图 5.6 融合操作策略测试结果

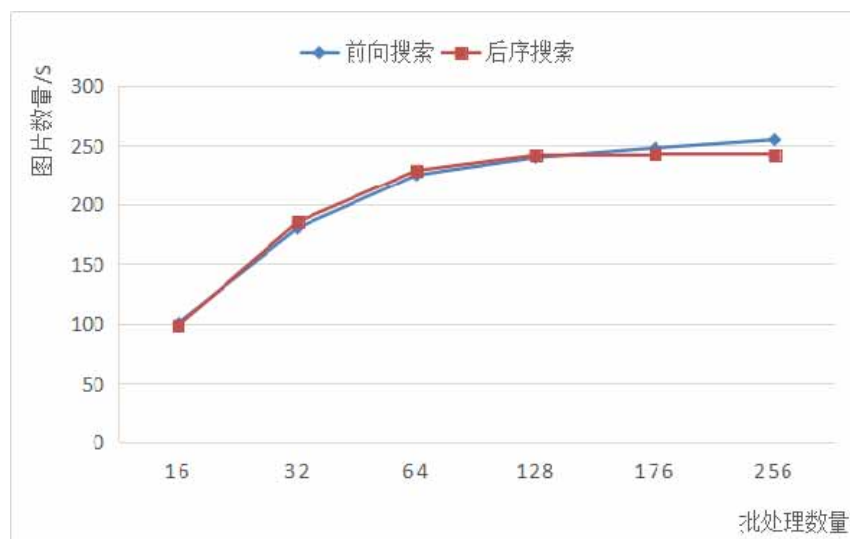


图 5.7 两种搜索策略实验结果

对于两种不同的实现方案，本文的实验验证得到了如图 5.7 所示的结果，该结果与理论分析的结果吻合，两种不同的实现方案的区别在于，一个是从当前节点顺序向后查找合适的依赖添加节点，另一个是从后向的求导几点反向查找可添加依赖的节点，前者是最早的

情况，后者是最晚的情况，当系统的数据吞吐量和 GPU 上的运算性能很稳定是，两者没有区别，同样，两者的区别也在大批量的数据处理中开始体现。出现这种现象的原因是大的数据量导致系统中数据传输总线的过度负载，影响了系统的整体运算性能，在这个时候，更早的换回数据更能够保证后续的计算不收到影响，所以在性能上会略有提升。

（3）模型通用性验证

为了验证内存优化框架的通用性，使用 VGG-19 网络，在 mnist 数据集上做了测试，由于 mnist 数据集与原 TFLMS 模型的图片数据集相比，占用内存减小，所以本实验的测试中，系统的计算状态并没有达到饱和，所以折线整体较平稳。测试结果如下图 5.8、5.9 所示：

图 5.8 是训练 500 张图片，批处理为 10 张的实验结果，图 5.9 是训练 1000 张图片，批处理为 64 的实验结果，从图 5.8 中发现，受到系统的吞吐量，系统内存，系统计算量的影响，换出张量数为 150 和 1000 是折线的两个转折点，在 150 之前和 1000 之后性能较平稳。这种现象在图 5.9 中也有同样的体现，从两个实验结果中可以看出，本文的优化后的 TFLMS 框架，从换回代价的角度考虑，合理选择张量换回的触发节点，使得模型的性能维持在稳定的状态，即使在换出张量数量很大的情况下，也能保持较好的性能。

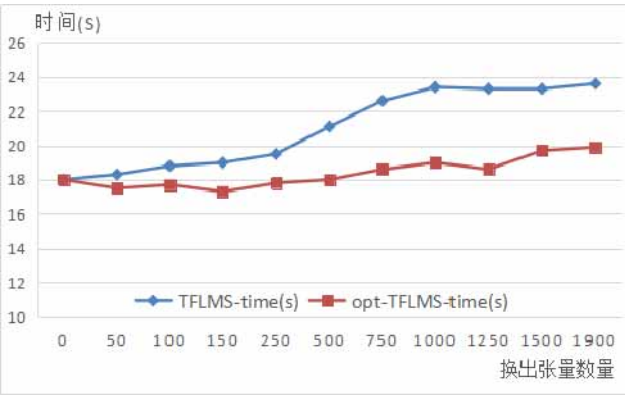


图 5.8 (500,10)模型训练时间

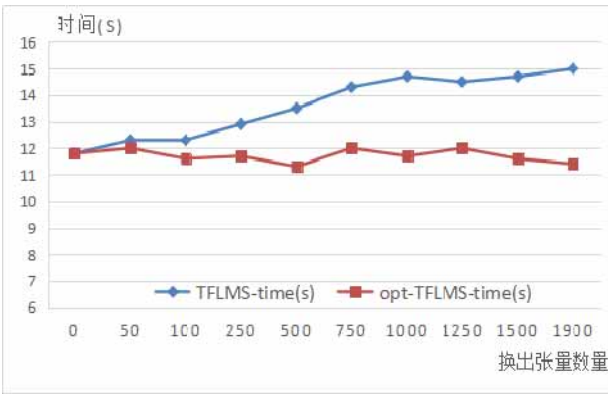


图 5.9 (1000,64)模型训练时间

对本文的基于代价的数据流优化模型的实验数据分析发现，优化模块在原模型的基础上性能整体提升约 4.6%。不同的实现方案在不同情况下体现优势，比如，融合存储优化策略在最开始的范围内有优势，随着批数量的不断增加，前向搜索方法有性能提升的趋势。这两种改进策略可依据实际情况选择，针对不同的网络模型，本文的优化模型同样适用。

5.3 本章小结

本章的主要内容是实验方案和实验结果验证分析，首先，通过大量的实验测试验证了本文的数据传输模型的准确性，然后，使用 Tensorboard 工具对深度学习神经网络的计算过

程做全面剖析，验证了性能评价模型具有可行性，最后，通过对比实验和不同模型上的实验验证说明了本文的数据流优化策略具备有效性和通用性。

结 论

深度学习算法在人工智能领域的应用越来越广，如何高效地发挥出算法加速平台的计算优势，是当前亟待解决的关键问题。本文对深度学习算法中 GPU 的访存性能做了分析与优化，提出的性能评估方法和框架数据流优化方法，有很好的通用性和高效性。通过本文的研究，得出以下结论：

(1) 通过细粒度基准程序与嵌入式汇编相结合的方法，得到了 Pascal 架构的内存特征参数和优化访存的编程方法，全面的揭示了 GPU 内存微架构特点。通过基准测试发现，实际的内存访存周期和吞吐量并没有达到系统的预期，使用控制访问步长、将数据向量化、控制传输数据量等方式可以缩短数据的访问时延。

(2) 在了解底层架构的基础上，本文提出了面向深度学习的 GPU 微架构访存性能评估方法，该方法从理论分析到实际运行，能够更准确更全面地从访存的角度对深度学习算法性能做出评估，并指导算法优化思路。通过实验发现，改进的 LogGp 模型可获得不同数据量传输性能参考标准，Roofline 性能峰值模型与预运行方法对比更容易发现算法的性能缺陷，通过 Tensorboard 了解模型的深层架构，是实现和优化数据流交换模型的重要依据。

(3) 本文提出的基于代价的数据流交换方法，与不考虑代价的换出换入策略相比，有很明显的性能提升。在保证模型的运算性能的前提下，以交换内存的方式利用系统带宽，提高 GPU 数据访存性能，是对 TFLMS 框架的很大的改进。本文中改进的融合操作和两种搜索方式在不同程度上起到了优化效果，可依据实际运行情况选择不同的优化策略。

在本文的研究过程中，依然发现很多的问题，需要今后进一步的研究。首先，目前在 GPU 上的基准测试方法依然沿袭之前的 P-chase 测试程序，并没有成型的辅助工具，如果了解底层架构，需要消耗时间来做统计分析工作，本文研究的下一步，希望能够在更新换代的 GPU 上形成通用的基准测试工具，辅助开发人员做上层应用的优化；其次，就当前流行的深度学习优化框架来看，很少有基于访存性能出发来做性能优化，TFLMS 数据流交换框架初步提出，并不是很完善，所以本文的下一步工作希望在加入计算代价的基础上加入内存，带宽等限制条件，更加精确的控制 GPU 上的访存和计算量，保证 GPU 达到更理想的计算性能。

参考文献

- [1] 李景军, 张宸, 曹强. 面向训练阶段的神经网络性能分析. 计算机科学与探索, 2018, 12(10):119-131 页
- [2] Hwu W . What is ahead for parallel computing. Journal of Parallel & Distributed Computing, 2014, 74(7):2574-2581P
- [3] Keckler S W , Dally W J , Khailany B , et al. GPUs and the Future of Parallel Computing. IEEE Micro, 2011, 31(5):7-17P
- [4] Ryoo S, Rodrigues C I, Bagsorkhi S S, et al. Optimization principles and application performance evaluation of a multithreaded GPU using CUDA. acm sigplan symposium on principles and practice of parallel programming, 2008: 73-82P
- [5] Lal S , Lucas J , Andersch M , et al. GPGPU workload characteristics and performance analysis. International Conference on Embedded Computer Systems: Architectures. IEEE, 2014: 115-124P
- [6] Ronan Ct, Koray K, Clement F. Torch7: A matlab-like environment for machine learning. In NIPSW, 2011: 5-10P
- [7] Bastien, F, Lamblin P , Pascanu R , et al. Theano: new features and speed improvements. Computer Science, 2012: 1121-5590P
- [8] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional Architecture for Fast Feature Embedding. acm multimedia, 2014: 675-678P
- [9] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition. computer vision and pattern recognition, 2016: 770-778P
- [10] Bahdanau D, Cho K, Bengio Y, et al. Neural Machine Translation by Jointly Learning to Align and Translate. international conference on learning representations, 2015:0473-1409P
- [11] Minh T, Hieu P, Christopher D . Effective approaches to attention-based neural machine translation. In Conferenceon Empirical Methods in Natural LanguageProcessing, 2015:1508-4025P
- [12] 裴颂文, 吴小东, 唐作其. 异构千核处理器系统的统一内存地址空间访问方法. 国

防科技大学学报, 2015, 37(1):28-33 页

- [13] Shirahata K, Tomita Y, Ike A, et al. Memory reduction method for deep neural network training. international workshop on machine learning for signal processing, 2016: 1-6P
- [14] Rhu M, Gimelshein N, Clemons J, et al. Virtualizing Deep Neural Networks for Memory-Efficient Neural Network Design. arXiv: Distributed, Parallel, and Cluster Computing, 2016: 43P
- [15] Chen M, Minmin S, Jun Y, Minghui Q. Training deeper models by GPU memory optimization on TensorFlow. In Proc. of ML Systems Workshop in NIPS, 2017
- [16] Rhu M, Gimelshein N, Clemons J, et al. vDNN: virtualized deep neural networks for scalable, memory-efficient neural network design. international symposium on microarchitecture, 2016: 1-13P
- [17] Le T , Imai H , Negishi Y , et al. TFLMS: Large Model Support in TensorFlow by Graph Rewriting. 2018: 1807-2037P
- [18] Chen T, Xu B, Zhang C, et al. Training Deep Nets with Sublinear Memory Cost. arXiv preprint arXiv. 2016: 1604-6174P
- [19] 张函. 基于 GPU 的深度神经网络模型并行及优化方法研究. 华中科技大学, 2016
- [20] Zhang Y, Owens J D. A quantitative performance analysis model for GPU architectures. high-performance computer architecture, 2011: 382-393P
- [21] Hong S, Kim H. An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness. international symposium on computer architecture, 2009, 37(3): 152-163P
- [22] Bagsorkhi S S, Delahaye M, Patel S J, et al. An adaptive performance modeling tool for GPU architectures. acm sigplan symposium on principles and practice of parallel programming, 2010, 45(5): 105-114P
- [23] Williams S, Waterman A, Patterson D A, et al. Roofline: an insightful visual performance model for multicore architectures. Communications of The ACM, 2009, 52(4): 65-76P
- [24] 江慧芳, 蔡达, 王晓蕊. 基于 CPU-GPU 异构环境的运算代价评估模型. 计算机工程, 2017, 43(9): 12-16 页
- [25] Czechowski K, Battaglini C, Mcclanahan C, et al. On the communication complexity

- of 3D FFTs and its implications for Exascale. international conference on supercomputing, 2012: 205–214P
- [26] GÓMEZ-LUNA J, GONZÁLEZ-LINA RESB J M. Performance Models for Asynchronous Data Transfers on Consumer Graphics Processing Units. Journal of Parallel and Distributed Computing, 2012, 72(9) :1117–1126P
- [27] 盛冲冲, 胡新明, 李佳佳, 等. 面向节点异构 GPU 集群的编程框架. 计算机工程, 2015, 41(2) : 292–297 页
- [28] Van B, Maassen J, Seinstra F J, et al. Performance Models for CPU–GPU Data Transfers. cluster computing and the grid, 2014: 11–20P
- [29] 苏华友. 面向应用的 GPU 并行计算关键技术研究. 国防科学技术大学, 2014
- [30] Jia W, Shaw K A, Martonosi M, et al. Characterizing and improving the use of demand–fetched caches in GPUs. international conference on supercomputing, 2012: 15–24P
- [31] Xie X, Liang Y, Sun G, et al. An efficient compiler framework for cache bypassing on GPUs. international conference on computer aided design, 2013: 516–523P
- [32] Jang B, Schaa D, Mistry P, et al. Exploiting Memory Access Patterns to Improve Memory Performance in Data–Parallel Architectures. IEEE Transactions on Parallel and Distributed Systems, 2011, 22(1): 105–118P
- [33] Che S, Sheaffer J W, Skadron K, et al. Dymaxion: optimizing memory access patterns for heterogeneous systems. IEEE international conference on high performance computing data and analytics, 2011: 13P
- [34] Sung I, Stratton J A, Hwu W W, et al. Data layout transformation exploiting memory–level parallelism in structured grid many–core applications. international conference on parallel architectures and compilation techniques, 2010: 513–522P
- [35] Li C, Song S L, Dai H, et al. Locality–Driven Dynamic GPU Cache Bypassing. international conference on supercomputing, 2015: 67–77P
- [36] Brecht T B, Guha K. Using parallel program characteristics in dynamic processor allocation policies. Performance Evaluation, 1996, 27–28(7):519–539P
- [37] Saavedra–Barrera R H, Smith A J. Measuring Cache and TLB Performance and Their Effect of Benchmark Run. IEEE Transactions on Computers, 1995, 44(10):1223–1235P

- [38] Kim C, Park K H. Credit-Based Runtime Placement of Virtual Machines on a Single NUMA System for QoS of Data Access Performance. *Computers IEEE Transactions on*, 2015, 64(6):1633-1646P
- [39] Duchateau A, Sidelnik A, Garzaran M J, et al. P-Ray: A Software Suite for Multi-core Architecture Characterization. *languages and compilers for parallel computing*, 2008:187-201P
- [40] Volkov V, Demmel J. Benchmarking GPUs to tune dense linear algebra. *ieee international conference on high performance computing data and analytics*, 2008: 1-11P
- [41] Fang M, Fang J, Zhang W, et al. Benchmarking the GPU memory at the warp level. *parallel computing*, 2018: 23-41P
- [42] Wong H, Papadopoulou M, Sadooghialvandi M, et al. Demystifying GPU microarchitecture through microbenchmarking. *international symposium on performance analysis of systems and software*, 2010: 235-246P
- [43] Bagsorkhi S, Gelado I, Delahaye M, et al. Efficient performance evaluation of memory hierarchy for highly multithreaded graphics processors. *acm sigplan symposium on principles and practice of parallel programming*, 2012, 47(8): 23-34P
- [44] Mei X, Zhao K, Liu C, et al. Benchmarking the memory hierarchy of modern GPUs. *IFIP International Conference on Network and Parallel Computing*. Springer, Berlin, Heidelberg, 2014: 144-156P
- [45] Zhang Y, Owens J D. A quantitative performance analysis model for GPU architectures *IEEE International Symposium on High Performance Computer Architecture*. IEEE Computer Society, 2011:383-393P
- [46] 陈睿, 杨孟飞. 航天嵌入式软件数据访问冲突基准测试集研究. *中国空间科学技术*, 2017: 3 页
- [47] Mei X, Chu X. Dissecting GPU Memory Hierarchy Through Microbenchmarking. *IEEE Transactions on Parallel and Distributed Systems*, 2017, 28(1): 72-86P
- [48] 胡冰. PCI-E 3.0 简介及信号和协议测试方法. *中国集成电路*, 2016, 25(12):67-74 页
- [49] Culler D E, Karp R M, Patterson D, et al. LogP: a practical model of parallel computation. *Communications of the Acm*, 1996, 39(11):78-85P

- [50] Alexandrov A , Ionescu M F , Schauser K E , et al. LogGP: Incorporating Long Messages into the LogP Model for Parallel Computation. *Journal of Parallel and Distributed Computing*, 1997, 44(1):71–79P
- [51] Kielmann T , Bal H E , Gorlatch S , et al. Network performance-aware collective communication for clustered wide-area systems. *Parallel Computing*, 2001, 27(11):1431–1456P
- [52] Jia Z, Maggioni M, Staiger B, et al. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. *arXiv: Distributed, Parallel, and Cluster Computing*, 2018: 1804-6826P
- [53] Zhao K, Chu X. G-BLASTN: accelerating nucleotide alignment by graphics processors. *Bioinformatics*, 2014, 30(10): 1384–1391P
- [54] Li Y, Chi H, Xia L, et al. Accelerating the scoring module of mass spectrometry-based peptide identification using GPUs. *BMC Bioinformatics*, 2014, 15(1): 121–121P
- [55] NVIDIA, matrix Mul, CUDA SDK 6.5, 2014
- [56] Chen M, Minmin S, Jun Y, Yang G. Training deeper models by gpu memory optimization on tensorflow. In *Proc. of ML Systems Work hop in NIPS*, 2017

攻读硕士学位期间发表的论文和取得的科研成果

参与的科研项目

- [1] 2016.09-2017.03 基于云计算的车载物联网系统
- [2] 2017.03-2018.09 面向 GPU 微体系结构汇编级优化研究

致 谢

时光荏苒，日月如梭，转眼间已到了 2019 年，我的硕士生涯也即将结束。回顾在哈尔滨工程大学两年多的硕士生活，我过的非常充实，这里有尽职尽责的老师，开朗热心的同学和朋友们，在项目中，我们在一起讨论问题、解决问题，在课余时间，我们聚餐，一起游玩，像一个大家庭一样，互相帮助和支持。借此机会，我想对我的老师、同学、朋友和家人们表达我最真挚的感谢。

我最感谢的是我的导师张国印老师。张老师学识渊博、平易近人，感谢张老师两年多来对我学习和生活上的指导和教诲。在学习上，张老师总能在宏观上为我把好方向，使我在每个学习阶段都不会迷茫。与张老师交流，我总能学习到严谨的治学态度，和端正的做人哲学，张老师总是以身作则，把自己的人生经验传授给我们，特别是在我择业的时候，给我最中肯的意见。在此向老师表示由衷的感谢，祝您身体健康，工作顺利！

同时我还要感谢吴艳霞老师，吴老师给我们提供了很好的项目实践平台，并且，在项目中给我们耐心的指导，使我掌握了扎实的专业技能，吴老师兢兢业业，踏踏实实的科研态度，给我很大的鼓舞，在以后的工作中我会向老师学习，秉承踏实勤奋的工作作风，在此，向老师表达我最真诚的祝愿，祝老师事业顺利，身体健康！

我还要感谢实验室的兄弟姐妹们。这些年，大家一起奋斗的时光是最美的记忆，感谢我的室友，在生活中给我欢乐和陪伴。

最后，要感谢我的家人，感谢你们一直以来对我的支持和关爱，你们是最坚强的后盾，我也会不负所望，在以后更加努力，更加优秀。